

A post-quantum Distributed OPRF from the Legendre PRF

Novak Kaluđerović
Zircuit
Zürich, Switzerland
kolja@zircuit.com

Nan Cheng
University of St. Gallen
St. Gallen, Switzerland
nan.cheng@unisg.ch

Katerina Mitrokotsa
University of St. Gallen
St. Gallen, Switzerland
katerina.mitrokotsa@unisg.ch

Abstract—A distributed OPRF enables a client to evaluate a pseudo-random function on a client-chosen input using a distributed key shared among multiple servers, ensuring that the servers learn nothing about the input or output, while the client learns nothing about the key. We present a post-quantum OPRF suitable for a distributed server setting, requiring only two rounds of communication between the client and servers, with server-to-server communication limited to a pre-computation phase. Our approach leverages the Legendre PRF, computed through a single MPC multiplication and opening during the online phase. We introduce a novel MPC technique that achieves multiplication and opening in one round using replicated secret sharing (RSS) in a malicious adversarial model. This allows for the quantum-secure, verifiable evaluation of the Legendre OPRF against malicious adversaries under a threshold assumption, without requiring inter-server communication. Beyond the Legendre PRF, this method is also of interest for general MPC operations. To our knowledge, our distributed OPRF (dOPRF) is the first post-quantum construction with these properties. We compare our approach to state-of-the-art MPC solutions and provide an implementation of our proposed dOPRF, benchmarking it against existing OPRF constructions. In our comparison, our dOPRF protocol demonstrates superior efficiency.

Index Terms—distributed OPRF, Legendre PRF, MPC

1. Introduction

An oblivious PRF (OPRF) is a two party protocol that involves two parties a client (receiver) and a server (sender) in which the server has a PRF key k and the client learns $F_k(x)$, where F is a PRF and x is the input chosen by the client. The design of the OPRF ensures that the server cannot gain any knowledge about the client's input or output, while it also prevents the client from learning anything about the server's key.

OPRFs were originally introduced by Freedman et al. [28], and they found widespread application in numerous areas of cryptography. The inherent obliviousness of OPRFs facilitates the development of protocols that safeguard private data. A common design paradigm to leverage an OPRF is to let a client compute a high-entropy cryptographic object (e.g., a token) from a low-entropy input (e.g., a password). Furthermore, the fact that the computation is assisted by one or multiple servers allows for delegating computations to the servers and keeping

a lightweight computation on the client side. The high-entropy cryptographic material can then be securely stored on the servers and does not need to be stored by the client, which only needs to remember a low-entropy value such as a password.

A variation of OPRFs are distributed OPRFs (dOPRFs), originally introduced by Jarecki et al. [38], which consider the setting where multiple servers are employed for the evaluation of the function. The secret key k is distributed among the servers and the servers and the client jointly evaluate the function $F_k(x)$ on the client's input x without revealing any information about x to the servers. Additionally, the distributed nature of the key provides additional protection from server breaches.

OPRFs have a wide range of applications and are a fundamental building block for creating password-authenticated key exchange [2], [3], [53], password secret sharing [10], [21], [36]–[38], single sign-on protocols [4], [10], and private set intersection [34], [39], [40], [45], among many other use cases. All existing practical OPRF constructions base their security on the difficulty of solving the discrete logarithm problem. This reliance makes such constructions vulnerable to attacks by quantum computers. Post-quantum OPRFs are an active area of research, and current constructions are either broken [9], [12], inefficient [6], [8], only presented in the semi-honest server model [5], [22], [25], or require many rounds of communication [11], [12], and neither of them are distributed. While existing generic MPC protocols [16], [18] offer strong security guarantees, they come with multiple rounds of communication between the client and servers, which can lead to delays as the client waits for the servers to exchange data before receiving the final output.

Therefore, we address the following open question: *Is it possible to construct a distributed OPRF with optimized communication rounds?*

Our answer is affirmative and we introduce a distributed OPRF with optimized communication (two rounds) that provides post-quantum security guarantee in the fully malicious model.

Contributions. Our contributions are as follows:

(i) *We propose a 2-round MPC protocol that computes the Legendre PRF in the fully malicious adversarial model.* More precisely, in our proposed OPRF the client and the servers run a MPC protocol to compute $(x, k) \mapsto (F_k(x), \perp)$, where the client shares their input with the servers and immediately receives the output, resulting in just two communication rounds. Unlike traditional MPC methods, our protocol allows the client to communicate

with each server in parallel, eliminating the need for inter-server communication and reducing potential bottlenecks. Contrary to existing MPC frameworks [16], [18] that would require multiple communication rounds, our proposed OPRF requires optimal communication rounds (two rounds), since it relies on a novel *verifiable, non-interactive malicious secure multiply-and-open* MPC protocol.

At a high level, our method can be understood as an MPC approach where servers compute the PRF using the circuit $C_{\text{PRF}} : (x, k, s^2) \mapsto (x + k)s^2$, as depicted in Fig. 1. The setup phase involves a single round of inter-server MPC communication to generate auxiliary elements needed for the online phase. After this setup, the online phase consists of two rounds of direct communication between the client and each server, executed in parallel.

We specifically use the Legendre PRF C_{PRF} as our candidate due to its efficiency in MPC, requiring only two MPC multiplications. The combination of this lightweight PRF and our MPC approach allows the PRF to be computed in just one communication round between the client and servers, bypassing the need for inter-server communication during the online phase. This leads to several advantages, including improved efficiency, reduced communication costs, and a simpler protocol.

(ii) *The core component is a verifiable non-interactive maliciously secure MPC multiply-and-open protocol, which is of independent interest.* It can either be used for general MPC operations or other applications (e.g., could be used to compute the inner product between two vectors, for instance in biometric authentication).

(iii) *We provide a comparison to existing MPC solutions, and benchmarkings against similar classical and post-quantum OPRFs.* Our proposed OPRF is rather efficient for a small number of servers, ≤ 7 which is usually the case in distributed OPRF scenarios. Our protocol though scales exponentially with increasing threshold and employed number of servers. Nevertheless, since the realistic setting of evaluating an OPRF would require less than 7 servers, the required efficiency is achieved.

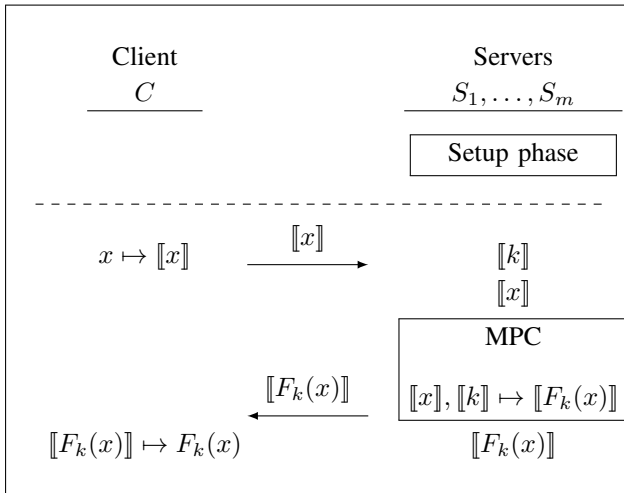


Figure 1: MPC based distributed OPRF

Adversarial model. We design our protocols in the presence of malicious adversarial behaviors. We call an adversary *malicious* if they have the full freedom to run any

strategy available to them. The adversary is assumed to be probabilistic polynomial-time.

The main players in the protocol are the client and n servers. We denote by t the threshold on the number of corrupted servers. The client may or may not be corrupted irrespective of t . We provide protocols that are provably secure when $t < n/3$ in the malicious setting.

The non-corrupted servers are *honest*, meaning that they follow the protocol and do not collude with any corrupted parties.

Our proposed OPRF provides *output verification*, meaning that the client can be assured of the output's correctness. We also introduce the concept of *input verification*, which is crucial in the new MPC setting. This protects the honest servers from the adversary in the event of malicious client behavior. Specifically, it ensures that the servers do not leak any sensitive data when the client behaves maliciously. In such cases, the client is given a random output.

We assume that the communication between the client and the servers, as well as the inter-server communication during the pre-computation stage, occurs over a secure channel. We also assume that the servers hold some secret shares which are used as randomness seeds. This is similar to a common reference string, but instead of a common string the parties hold a secret sharing, i.e., some subsets of parties hold common strings. These seeds can also be generated by means of a non-interactive key exchange (NIKE) method. The seeds are then used for generating random values in the setup stage of the protocol.

Lastly, the adversary is assumed to be static, meaning that the set of corrupted parties is determined at the beginning of the protocol.

Technical framework. The Legendre PRF can be computed using simple field arithmetic. Computing the Legendre PRF function $F_k(x)$ is information-theoretically equivalent to revealing a field value $(x + k)s^2$ where s is a random non-zero field element unknown to the client or any server. Thus, the Legendre PRF's computation involves performing one squaring, one addition, and one multiplication.

We use replicated secret sharing (RSS) for this computation, where each field element x, k, s^2 is stored as an RSS share among the servers. Due to the scheme's linearity, the computation of addition is straightforward. For the remaining operations, we identify two different operations: *RSS to RSS multiplication* and *multiplication-with-opening*, one for the computation of the square in the offline phase and the other for the online evaluation. For the latter operation, we proposed two variants to realize it depending on if the client colludes with the corrupted servers.

In Section 3.3 we introduce *doubly replicated secret sharing* (DRSS), a secret sharing scheme which realizes the same access structure as RSS, but acts as a useful tool for computing multiplications, particularly when adding verification to our algorithms.

We provide an algorithm for computing the Legendre OPRF in a malicious setting (Section 5.2), which follows the structure shown in Fig. 1. There is a setup phase during which the servers compute a random square s^2 , various secret shares of zero and secret shares of a random number. This process requires a single round of communication.

The result is then stored for subsequent use in the online phase. In this phase, the servers receive an RSS of the input x from the client. They compute an addition $x + k$ and a multiplication with the stored square, revealing the result $(x + k)s^2$ to the client.

Both phases involved in our OPRF algorithm provide post-quantum computational security, with the only non-information-theoretic parts being hash functions and PRFs used for verification procedures.

Related work. Oblivious PRFs have been systematically studied by Casacuberta et al. [13]. The authors divide them into four main categories based on the underlying assumptions – Naor-Reingold (NR), Hashed Diffie-Hellman (HashDH and 2HashDH), and Dodis-Yampolskiy (DY), all of which are based on classical security assumptions. In Tab. 1 we compare some popular OPRFs against ours. *Partially oblivious* property means that the input provided by the client is split into a private part x_{priv} which remains secret, and public part x_{pub} which is revealed to the server(s). *Verifiability* allows the client to be convinced of the correctness of the received output. A *distributed* OPRF works in a multi-server setting (i.e., the secret key is distributed to multiple servers). In our case, the verifiability stems from the threshold assumption of a (super)majority of honest servers.

OPRFs that do not rely on the previously mentioned primitives are scarce. One such example is a lattice-based construction proposed by Albrecht et al. [6]. While this construction offers verifiability, it is notably inefficient, requiring more than 128GB of bandwidth for each evaluation. There are also two existing constructions based on isogenies. The first, proposed by Boneh et al. [12], is built on the CSIDH [14] framework. The same paper proposed an additional protocol based on SIDH [27], [35], which was found to be vulnerable to specific attacks. Basso [8] proposed a countermeasure and an enhanced algorithm to address these vulnerabilities. This improved construction also provides the advantage of verifiability.

Recently, Dodgson [23] modeled and proved the security of the Legendre OPRF in the Universal Composability (UC) model. They focused on the single server case, leading to security assumptions that are different from those in our scenario. The author implemented and benchmarked the OPRF using the MP-SPDZ [43] framework and the MASCOT [44] protocol, however in the malicious setting the protocol takes 14 rounds of communication in the online phase, and at least 92 rounds in the offline phase.

Recent work by Beullens et al. [11] provided a practical OPRF construction based on the Legendre PRF, together with a proof of security in the UC framework. Their construction relies on oblivious transfer and vector oblivious linear evaluation. This approach makes the protocol somewhat practical, requiring 1MB of total communication and an estimated evaluation time of 0.57 seconds on a WAN network with a latency of 30 ms and a throughput of 25 Mbps. However, the protocol requires sending 9 messages (rounds) between the server and client, with the communication itself being the bottleneck. At the moment no distributed version of the said protocol exists and the authors consider it as an open challenge for future research.

Faller et al. [25] show how to construct efficient, composable and quantum secure OPRFs from generic

techniques. They use garbled circuits (GC) to construct and prove the security of an OPRF in the UC model with malicious users and semi-honest servers. In addition to that they implement and benchmark two versions of their protocol, which we compare with in Tab. 1.

Albrecht et al. [5] provide a post-quantum OPRF based on torus fully homomorphic encryption. Their protocol provides security in the malicious client but only the semi-honest server model. The protocol is rather efficient at 2 rounds, 10KB of communication and 151ms, but the provided implementation does not contain the necessary zero-knowledge proofs, and the authors expect an additional overhead of $5\times$ to $10\times$ in terms of run-time.

For computing the Legendre PRF, with only one addition, one multiplication and opening in the online phase, it is possible to construct a dOPRF based on general malicious MPC framework like those in Dalskov et al. [18] or Chida et al. [16], but the resulting efficiency is not as good as the one of our proposed dOPRF. More precisely, [18] and [16] would require 3 and 9 rounds respectively to construct a dOPRF, while our protocol requires just 2 rounds. In section 6 we compare our protocol to theirs in full detail.

TABLE 1: Comparison of some state of the art OPRFs.

OPRF	PRF	Partially oblivious	Verifiable	Distributed	Quantum resistant
[34], [37]	NR	✗	✓	✗	✗
[39]	DY	✗	✓	✗	✗
[10], [36], [38]	2HashDH	✓	✗	✓	✗
[6], [12]	Isogenies	✗	✓	✗	✓
[25]	GC	✗	✗	✗	✓
[11], [23]	Legendre PRF	✗	✓	✗	✓
Our work	Legendre PRF	✓	✓	✓	✓

On distributed security. We note that our protocol offers different security properties in the distributed and threshold setting than previous classical constructions. Other dOPRFs preserve the secrecy of the input x when an adversary takes control of $> t$ servers. In our scenario a weaker security assumption holds – both the input x and the key k are revealed once a malicious party breaches a threshold of the servers.

While this may seem like a strong security relaxation, we argue that the benefits outweigh the drawbacks. A malicious party that takes full ($> t$ threshold) control of the servers always learns the secret key k . This allows for a brute-force dictionary attack which can be used to reveal users’ inputs. In general users’ inputs are passwords (or hashes thereof) which are not assumed to be of high-entropy, and this attack would reveal any such inputs. A quantum computer would speed these attacks even further by employing a faster search algorithm such as Grover’s search [33].

A second important property to note is that in our protocol the users’ inputs would be revealed *only if the attacker takes control of the servers’ during the protocol*. The same is not true of the keys, which are long-term secrets stored on the servers, and which would be revealed to an attacker in any case of a server’s breach. We emphasize though that our proposed dOPRF makes a significant trade-off and creates a very efficient protocol with only

a **round-trip communication** and very fast evaluation times.

2. Preliminaries

Notation. Let p be a prime. We assume that p is public and $p \approx 2^{2\lambda_c}$ or $p \approx 2^{3\lambda_q}$ where λ_c and λ_q are classical and quantum security parameters respectively. We denote with $\mathbb{F}_p$ the finite field of p elements. For any set S , we denote with $\#S$ the cardinality of S . We use subscripts to distinguish the terms of a sum ($a = \sum a_i$) or to index parties in a protocol (server S_i), while we use superscripts to denote values presumed to be equal and allocated to various parties (a^i given to party associated with index i).

Secret sharing. Secret sharing is a method for distributing a secret value $x \in \mathbb{F}_p$ among multiple parties, so no single party, or a predefined group of parties, holds any information about the secret. We call t a threshold, and a corresponding secret sharing scheme a (t, n) scheme, if any group of at most t colluding parties cannot extract any information about the secret, while any group of $\geq t + 1$ parties can compute the secret x .

We denote with $\llbracket x \rrbracket$ a generic secret sharing of a field element $x \in \mathbb{F}_p$, i.e., an n -tuple $(x_1, \dots, x_n)$ such that each share x_i is owned by party i .

Multi-Party Computation. Secure multi-party computation (or MPC) allows two or more parties to compute any function on private inputs without revealing anything but the output of the function. We consider only arithmetic MPC protocols, where the parties hold (t, n) secret sharings of some field elements, and their goal is to compute secret sharings of arithmetic functions of these elements, while preserving all security properties under the same threshold t .

In our setting (Figure 1), in addition to the n computing parties, there is a single client C which provides a part of the input to the computing parties, and receives the output of the computation. We only concentrate on the protocols which can efficiently compute field additions and multiplications. More precisely, we develop MPC techniques which, given some secret sharings $\llbracket x \rrbracket, \llbracket y \rrbracket$ compute $\llbracket x \cdot y \rrbracket$ and reveal $x \cdot y$ to the client, while preserving security properties even with malicious clients, and without the need for any inter-server communication.

Oblivious PRFs. A *Pseudorandom Function* (PRF) family is a collection of functions $\{F_k\}_{k \in \mathcal{K}}$ with the same domain and co-domain, indexed by some key set $\mathcal{K}$, where a randomly sampled function from this family is difficult to be distinguished from a truly random function with the same domain and co-domain.

An *Oblivious Pseudorandom Function* (OPRF) is a two-party protocol between a client and a server. It enables the server to evaluate a pseudorandom function on an input chosen by the client using the server's key. The protocol ensures that the server cannot learn anything about the client's input or output, while also preventing the client from learning anything about the server's key.

An OPRF is called *distributed* (dOPRF) if there are multiple servers taking the role of the OPRF evaluator. The key is assumed to be distributed between the servers by means of secret sharing. In this scenario, multiple servers evaluate a PRF in a multi-party computation

setting, ensuring the confidentiality of the key and input/output values. A distributed (t, n) OPRF (dOPRF) is a sub-type of an OPRF which can withstand t malicious colluding servers.

2.1. The Legendre PRF

The usage of Legendre symbols in a pseudorandom function is an idea originally proposed by Damgård [19]. Due to its impractical nature, being orders of magnitude slower than generic PRF counterparts, it did not gain much attention until a result by Grassi et al. [32] sparked an interest in the Legendre PRF [26] because it was found suitable as a multi-party computation friendly PRF. This is mainly due to the homomorphic property of the Legendre symbol and the possibility of evaluating it with only three modular multi-party multiplications, which makes it a very efficient MPC friendly PRF candidate.

Definition 1 (Legendre symbol). *We define the Legendre symbol by setting:*

$$\left(\frac{a}{p}\right) = a^{\frac{p-1}{2}} = \begin{cases} 1 & \text{if } a \in \mathbb{F}_p \text{ is a square mod } p \\ -1 & \text{if } a \in \mathbb{F}_p \text{ is not a square mod } p \end{cases}$$

In general, the Legendre symbol is defined by setting $\left(\frac{0}{p}\right) = 0$, which makes the symbol multiplicative, but comes at a cost of increasing the size of the co-domain. We will assume that $\left(\frac{0}{p}\right) = 1$ and that the symbol is multiplicative, which is a reasonable assumption, since for our use-case the inputs are random, and the probability that the input is $x = 0$ is negligible since the input is an element not known to the adversary. In particular, we assume

$$\left(\frac{a}{p}\right) \left(\frac{b}{p}\right) = \left(\frac{ab}{p}\right) \quad \text{for all } a, b \in \mathbb{F}_p. \quad (1)$$

Definition 2 (Legendre PRF). *We define the Legendre PRF to be the family of functions $\{F_k\}_{k \in \mathcal{K}}$ the key space being $\mathcal{K} = \mathbb{F}_p$, the functions parameterised as $F_k : \mathbb{F}_p \rightarrow \{-1, +1\}$ and defined as*

$$F_k(x) := \left(\frac{x+k}{p}\right).$$

Under this definition, the Legendre PRF outputs only a single bit. These functions can be expanded to a λ -bit output function by increasing the keyspace to $\mathcal{K}^\lambda$ and for each $k = (k_1, \dots, k_\lambda) \in \mathcal{K}^\lambda$ defining

$$F_k(x) := \left(\left(\frac{x+k_1}{p}\right), \left(\frac{x+k_2}{p}\right), \dots, \left(\frac{x+k_\lambda}{p}\right) \right).$$

Legendre PRF hardness assumptions. The two main problems conjectured to be hard and on which the security of the Legendre PRF is based are the following:

Definition 3 (Shifted Legendre Symbol Problem - SLSP). *Let k be a uniformly random value in $\mathbb{Z}/p$. Given access to an oracle $\mathcal{O}$ that on input $x \in \mathbb{Z}/p$ computes $\mathcal{O}(x) = \left(\frac{k+x}{p}\right)$, find k .*

Definition 4 (Decisional Shifted Legendre Symbol Problem - DSLSP). *Let k be a uniformly random value in $\mathbb{Z}/p$. Let $\mathcal{O}_0$ be an oracle that on input $x \in \mathbb{Z}/p$ computes $\mathcal{O}_0(x) = \left(\frac{k+x}{p}\right)$, and let $\mathcal{O}_1$ be an oracle that on input x outputs a random value in $\{-1, +1\}$. Given access to $\mathcal{O}_b$ where b is an unknown random bit, find b .*

It is clear that the SLSP is at least as hard as DSLSP, so $\text{DSLSP} \leq \text{SLSP}$ under polynomial time reductions. The best known attack on DSLSP [41], [42] has a run-time of $O(\sqrt{p} \log \log p)$. Specifically, if the attacker is restricted to M queries to the oracle, the attack takes $O(\frac{p \log p \log \log p}{M^2})$ time and requires $O(M^2 \log p)$ memory. The optimal run-time is achieved at $M = \sqrt[4]{p \log \log p \log p}$.

In context of quantum computing, a polynomial time attack exists, when quantum access to the oracle is provided [51]. However, this scenario is not deemed realistic. In our case, and all general dOPRF settings, the servers are classical computers that respond to classical queries with classical outputs, which is modeled with a classical-query oracle. The best known quantum attacks with classical oracle queries are still exponential and they require $M = \tilde{O}(\sqrt[3]{p})$ queries, M^2 classical memory with quantum random access, and achieve an $\tilde{O}(\sqrt[3]{p})$ run-time by means of Grover's [33] or Tani's [50] search algorithms. An attacker with quantum memory can launch an attack with $M = \sqrt[3]{p}$ classical queries, $2^{O(\sqrt{\log_2 p})} \sqrt[3]{p}$ run-time and $2^{O(\sqrt{\log_2 p})}$ quantum memory [30].

All these attacks can directly be generalised to the λ -bit Legendre PRF. This is easy to see as the multi-bit PRF is simply a concatenation of λ independent single-bit PRFs with i.i.d. keys k_i . Specifically, there are polynomial-time reductions with at most a multiplicative $O(\lambda)$ overhead, which reduces the multi-bit (D)SLSP to (D)SLSP and vice-versa.

This implies that according to our best assumptions, a λ -bit Legendre PRF over a prime p of size 2λ provides $\lambda_c = \lambda$ bits of classical security and $\lambda_q = 2/3\lambda$ bits of quantum security. This is our main hardness assumption, and based on it we implemented a Legendre PRF of λ bits of classical and $2/3\lambda$ bits of quantum security over a prime of 2λ bits and with output length of λ bits, for $\lambda \in \{32, 64, 96, 128, 256\}$.

Multi-party Legendre symbol computation. The Legendre symbol $(\frac{a}{p})$ of a secret shared input $\llbracket a \rrbracket$ can be computed in an MPC setting as shown in Algorithm 1.

Algorithm 1 Legendre symbol computation

- 1 Sample a random $\llbracket s \rrbracket$ and compute $\llbracket s^2 \rrbracket$
 - 2 Compute $\llbracket a \rrbracket, \llbracket s^2 \rrbracket \mapsto \llbracket as^2 \rrbracket$ and reveal the outcome
 - 3 Compute the Legendre symbol

$$\left(\frac{as^2}{p}\right) = \left(\frac{a}{p}\right) \left(\frac{s^2}{p}\right) = \left(\frac{a}{p}\right)$$
-

Note that revealing as^2 is equivalent to revealing $(\frac{a}{p})$, since masking a with a random non-zero square gives us the same information as the Legendre symbol of a . The case of $s = 0$ is ignored due to its probability being negligible in the field size and thus, the security parameter.

The Legendre PRF has two inputs, x and k , and outputs the Legendre symbol of their sum $(\frac{x+k}{p})$. The inputs x and k are owned by the client and the servers correspondingly. The computation of the PRF on the client side is done by the client sharing their input as $\llbracket x \rrbracket$, and receiving the output at the end of the protocol. The servers compute an addition $\llbracket a \rrbracket = \llbracket x + k \rrbracket$, and execute Algorithm 1, while revealing the output only to the client.

For a λ -bit output, Algorithm 1 is repeated for each bit of the output. In particular λ keys $k_1, \dots, k_\lambda$ and λ random values $s_1, \dots, s_\lambda$ are computed for each of the λ output bits, and this procedure can be parallelised.

The security of the Legendre PRF implicitly depends on the number of queries that the attacker is allowed to make on the PRF oracle, as per Definitions 3 and 4. In the dOPRF setting, a client interacts with multiple servers in a MPC framework in order to query the PRF. We demonstrate that in the event of any malicious behaviour, either by the server or the client, the value reconstructed by the client will be random (as opposed to a queried PRF value). On the other hand, if everyone acts honestly, the client can reconstruct the PRF output in a manner that preserves the confidentiality of the key, input and output from the client and servers accordingly. Due to the privacy preserving property of the multiplication protocol, and the quantum-safety of the Legendre PRF, the quantum resistance of the Legendre dOPRF follows.

3. Arithmetic MPC and Secret Sharing

In this section, we recall the definitions of some basic secret sharing methods and arithmetic protocols. We call the parties which hold secret shares and are involved in MPC *servers*. We use the terms *server* and *server index* interchangeably to refer to a server $S_1, \dots, S_n$ or the specific index $i = 1, \dots, n$ assigned to that server. We also use the term *share* and *share index* interchangeably to refer to a share of a secret sharing held by some servers, and the index of the said share. For more detailed information the reader is advised to refer to [17], [24], [46].

3.1. Additive secret sharing

Additive secret sharing (ASS) is a method to share a field element $x \in \mathbb{F}_p$ among at least two servers indexed by some set $I \subseteq \{1, \dots, n\}$, with the security threshold $t = \#I - 1$. Any $\leq t$ colluding parties have no information about x .

Definition 5 (Additive secret sharing). *For $I \subseteq \{1, \dots, n\}$, $\#I > 1$, an element $x \in \mathbb{F}_p$ is additively secret shared among servers S_i for $i \in I$ as follows: Let $\mathcal{T} = I$ be the set of share indices. Select any share index $T_0 \in \mathcal{T}$ and then for each $T \in \mathcal{T} \setminus \{T_0\}$ sample x_T uniformly at random from $\mathbb{F}_p$, and set $x_{T_0} = x - \sum_{T \in \mathcal{T} \setminus \{T_0\}} x_T$. Then*

$$x = \sum_{T \in \mathcal{T}} x_T,$$

and the share of server S_i is the term x_T where $T = i$. We denote an ASS secret sharing of x with $[x]$.

3.2. Replicated secret sharing

Replicated secret sharing (RSS) [1] is a generalisation of additive secret sharing, in which the same additive secret shares of an element are provided to multiple parties. This approach provides more freedom in choosing the threshold $1 \leq t \leq n - 1$, which in turn enables

enhanced functionality. We assume that $t < \frac{n}{2}$, which permits communicationless multiplication. We follow the notation of [7].

Definition 6 (Replicated secret sharing). *The (t, n) -Replicated secret sharing scheme among n servers is defined as follows: The set of share indices denoted as $\mathcal{T}$ is:*

$$\mathcal{T} := \{A \subseteq \{1, \dots, n\} \mid \#A = t\},$$

and an element $x \in \mathbb{F}_p$ is secret shared by constructing an additive secret sharing of size $\#\mathcal{T} = \binom{n}{t}$, i.e.

$$x = \sum_{T \in \mathcal{T}} x_T$$

where all but one addends are sampled uniformly at random from $\mathbb{F}_p$, and the last one is selected so that the equation above is satisfied. The shares held by server S_i are exactly those x_T of index T where $i \notin T$.

An RSS secret sharing of a field element x is denoted with $\llbracket x \rrbracket_R$.

We define $\mathcal{T}_i \subseteq \mathcal{T}$ to be the set of shares held by party S_i , i.e.,

$$\mathcal{T}_i := \{T \in \mathcal{T} \mid i \notin T\}.$$

We further define the shares held by any pair of servers S_i, S_j as

$$\mathcal{T}_{ij} := \mathcal{T}_i \cap \mathcal{T}_j.$$

Each server S_i holds exactly $\binom{n-1}{t}$ shares.

We define $\mathcal{S}_T \subseteq \{1, \dots, n\}$ to be the set of indices of servers holding the share T :

$$\mathcal{S}_T := \{i \in \{1, \dots, n\} \mid T \in \mathcal{T}_i\} = \{1, \dots, n\} \setminus T.$$

We further define the set of servers holding any pair of shares T_1, T_2 as:

$$\mathcal{S}_{T_1 T_2} := \mathcal{S}_{T_1} \cap \mathcal{S}_{T_2}.$$

Each share T is held by exactly $n - t$ servers.

By abuse of notation, we denote with $\llbracket x \rrbracket_R$ the set of shares distributed to the servers by a (possibly corrupt) dealer.

$$\llbracket x \rrbracket_R = ((x_T^i)_{T \in \mathcal{T}_i})_{i=1, \dots, n}.$$

Each server S_i receives $(x_T^i)_{T \in \mathcal{T}_i}$ where x_T^i is the share associated to index T . We call $\llbracket x \rrbracket_R$ *valid* if $x_T^i = x_T^j$ for all $i, j \in \mathcal{S}_T$ and all $T \in \mathcal{T}$. Then $\llbracket x \rrbracket_R$ is a replicated secret sharing $\llbracket x \rrbracket_R$ of $x = \sum_{T \in \mathcal{T}} x_T^i$. Otherwise the sharing is called *invalid*, and it does not correspond to an RSS sharing of a field element.

Theorem 1 (t-Privacy of Replicated Secret Sharing). *Let $\llbracket x \rrbracket_R$ be an RSS of x distributed to n parties. For any subset of parties $\mathcal{S}$, with $|\mathcal{S}| \leq t$, the following conditions hold:*

- 1) *The view of the parties in $\mathcal{S}$ reveals no information about the secret x . Formally, for any secret x and any possible shares $s_1, s_2, \dots, s_t$ held by the parties in $\mathcal{S}$,*

$$\mathbb{P}[x \mid (x_T^1)_{T \in \mathcal{T}_1}, (x_T^2)_{T \in \mathcal{T}_2}, \dots, (x_T^t)_{T \in \mathcal{T}_t}] = \mathbb{P}[x].$$

- 2) *The shares of the parties in $\mathcal{S}$ can be perfectly simulated. That is, there exists a simulator Sim that*

generates shares $(\hat{x}_T^1)_{T \in \mathcal{T}_1}, (\hat{x}_T^2)_{T \in \mathcal{T}_2}, \dots, (\hat{x}_T^t)_{T \in \mathcal{T}_t}$ indistinguishable from the actual shares, without knowledge of the secret s .

3.3. Doubly replicated secret sharing

In this section, we introduce doubly replicated secret sharing (DRSS), a variation of RSS with the same access structure but a different distribution of the shares, so it naturally satisfies the same security and privacy properties of RSS, in particular the (t, n) -DRSS satisfies t -privacy and t -security just like RSS.

Definition 7 (Doubly replicated secret sharing). *The (t, n) -Doubly replicated secret sharing scheme among n servers is defined similarly to RSS, with the exception that the set $\mathcal{T}^2$ is used instead of $\mathcal{T}$ as the set of share indices.*

An element $x \in \mathbb{F}_p$ is shared by constructing an additive secret sharing of size $\#\mathcal{T}^2 = \binom{n}{t}^2$,

$$x = \sum_{T_1, T_2 \in \mathcal{T}} x_{T_1 T_2},$$

where all but one addends are sampled uniformly at random from $\mathbb{F}_p$, and the last one is selected so that the equation above is satisfied. The shares held by server S_i are exactly those $x_{T_1 T_2}$ where $i \notin T_1 \cup T_2$.

Using the same notation from Section RSS 3.2 we can see that each server S_i holds shares $\{x_{T_1 T_2} \mid T_1 \in \mathcal{T}_i, T_2 \in \mathcal{T}_i\}$, and that the share $x_{T_1 T_2}$ is held exactly by servers $\mathcal{S}_{T_1 T_2}$. In particular each $x_{T_1 T_2}$ is held by at least 1 server when $t < n/2$. Furthermore, as that $n - 2t \leq \#\mathcal{S}_{T_1 T_2}$ so when $t < n/3$, each share is held by at least $t + 1$ servers, in particular by at least 1 *honest* server.

By abuse of notation, we denote with $\llbracket x \rrbracket_D$ the set of shares distributed to the servers by a (possibly corrupt) dealer.

$$\llbracket x \rrbracket_D = ((x_{T_1 T_2}^i)_{T_1, T_2 \in \mathcal{T}_i})_{i=1, \dots, n}.$$

Each server S_i receives $(x_{T_1 T_2}^i)_{T_1, T_2 \in \mathcal{T}_i}$ where $x_{T_1 T_2}^i$ is the share associated to index $T_1 T_2$. We call $\llbracket x \rrbracket_D$ *valid* if $x_{T_1 T_2}^i = x_{T_1 T_2}^j$ for all $i, j \in \mathcal{S}_{T_1 T_2}$ and all $T_1, T_2 \in \mathcal{T}$. Then $\llbracket x \rrbracket_D$ is a doubly replicated secret sharing $\llbracket x \rrbracket_D$ of $x = \sum_{T_1, T_2 \in \mathcal{T}} x_{T_1 T_2}^i$. Otherwise the sharing is called *invalid*, and it does not correspond to a DRSS sharing of a field element.

3.4. Security Proof in MPC

We use simulation based proofs throughout the paper to argue about the security of our protocols. We consider two scenarios: the *real-world execution* and the *ideal-world execution*. In both scenarios, an adversary, denoted as $\mathcal{A}$, may control certain participants - this could be the client, some servers, or even a combination of the client and several servers. The adversary $\mathcal{A}$ has access to the input/output information of the parties under its control, potentially including intermediate random tapes or the ultimate output of the protocol.

In the ideal world, we define an ideal functionality which is deemed perfectly secure in the presence of the adversary $\mathcal{A}$. Meanwhile in the real world, we claim a protocol as perfectly (or statistically) secure if, for any given

adversary $\mathcal{A}$, there exists a PPT simulator capable of generating an output distribution, using only the transcripts available in the ideal-world execution, that perfectly (statistically) simulate views that are indistinguishable from the real-world protocol execution distribution.

4. Building Blocks and Sub-Protocols

In this section, we define the basic protocols that are used as subroutines during the OPRF computation. We use the following notation for share indices, $\hat{\mathcal{T}} \subseteq \{1, \dots, n\}$ (ASS), $\hat{\mathcal{T}} = \mathcal{T}$ (RSS) or $\hat{\mathcal{T}} = \mathcal{T}^2$ (DRSS), and the corresponding server set ($I \subseteq \hat{\mathcal{T}}$ for ASS or $\{1, \dots, n\}$ for RSS/DRSS). We assume that all protocols work in the malicious setting with the RSS-(t,n) setting, where there are no more than t servers denoted as $\mathcal{S}_{\text{Mal}}$ controlled by an adversary $\mathcal{A}$ that arbitrarily disrupts protocol execution, no less than $n - t$ servers denoted as $\mathcal{S}_{\text{Hon}}$ honestly follow protocol description.

4.1. Generating random secret shares

Random secret shares are used both as common reference strings shared among different parties as well as to mask and hide data and secret shares.

Random secret sharing. Let x be an element we want to compute a secret sharing of, and let s be a randomness seed. We denote with $(x_{\hat{\mathcal{T}}})_{\hat{\mathcal{T}} \in \hat{\mathcal{T}}} \leftarrow \text{ASS/RSS/DRSS}(x; s)$ the process of computing a random ASS/RSS/DRSS secret sharing of x . We assume that it is computed deterministically by using some PRF function F , for example as follows:

$$x_{\hat{\mathcal{T}}} = F_s(x, \hat{\mathcal{T}}),$$

$$x_{\hat{\mathcal{T}}'} = x - \sum_{\hat{\mathcal{T}} \in \hat{\mathcal{T}} \setminus \{\hat{\mathcal{T}}'\}} x_{\hat{\mathcal{T}}},$$

where $\hat{\mathcal{T}}_0$ is a predetermined share in $\hat{\mathcal{T}}$, for example the last one.

Secret sharing of a random number. We define $\mathcal{F}_{\text{GenRand}}$ as the ideal functionality that receives a command from a group of servers, and outputs corresponding shares of a random secret sharing of a random field element to each server, depending on the command, the output can be shares of a n-out-of-n additive secret sharing, a (t,n)-replicated secret sharing or a doubly replicated secret sharing. This functionality can be realized without communication between the servers, and the protocol generating these shares is provided in Appendix A.1.

Secret sharing of zero. We denote with $\llbracket z \rrbracket = (z_{\hat{\mathcal{T}}})_{\hat{\mathcal{T}} \in \hat{\mathcal{T}}}$ an additive secret sharing of zero. In particular, we have $\sum_{\hat{\mathcal{T}} \in \hat{\mathcal{T}}} z_{\hat{\mathcal{T}}} = 0$. Secret shares of zero are useful for re-randomising secret shares held by servers as $\llbracket x \rrbracket + \llbracket z \rrbracket = \llbracket x \rrbracket$. Secret sharing of zero can be computed during the pre-computation stage and saved for later use in the online stage. We define $\mathcal{F}_{\text{GenZero}}$ as the ideal functionality that receives a command from a group of servers and outputs corresponding shares of a random secret sharing of zero to each server. $\mathcal{F}_{\text{GenZero}}$ can also be realized without communication between the servers, as shown in Appendix A.2.

Secret sharings of zero are used to mask a reveal of a secret sharing. If the servers wish to reveal x to the

client, without revealing the shares $x_{\hat{\mathcal{T}}}$, they can reveal secret shares $x_{\hat{\mathcal{T}}} + z_{\hat{\mathcal{T}}}$ which are independent of the original shares, but open to the same value x .

4.2. Secret share addition

All previously mentioned secret sharing schemes are linear. The computation of $\llbracket a + b \rrbracket$ from $\llbracket a \rrbracket$ and $\llbracket b \rrbracket$ can be computed with local operations at each server by adding the corresponding shares of the addends to obtain the share of the sum, $a_{\hat{\mathcal{T}}} + b_{\hat{\mathcal{T}}} = (a + b)_{\hat{\mathcal{T}}}$.

4.3. RSS to RSS Multiplication

The first multiplication in Fig. 1 is computed in the setup phase where the servers compute $\llbracket s \rrbracket_R, \llbracket s \rrbracket_R \mapsto \llbracket s^2 \rrbracket_R$, so both the inputs and the output are RSS shares. We define the ideal functionality of $\mathcal{F}_{\text{Mul}}$ in Fig. 2.

Players: Server $\mathcal{S}_1, \dots, \mathcal{S}_n$ and the functionality $\mathcal{F}_{\text{Mul}}$.

Input: Every server $\mathcal{S}_i \in \mathcal{S}_{\text{Hon}}$ inputs their shares $(a_T, b_T)_{T \in \mathcal{T}_i}$ to $\mathcal{F}_{\text{Mul}}$, while each malicious server inputs whatever input to $\mathcal{F}_{\text{Mul}}$.

Computation: $\mathcal{F}_{\text{Mul}}$ checks if every RSS share submitted by each $\mathcal{S}_i \in \mathcal{S}_{\text{Mal}}$ is equal to the one corresponds to the one hold by $\mathcal{S}_{\text{Hon}}$, it outputs $\perp$ if there is any mismatch; otherwise $\mathcal{F}_{\text{Mul}}$ reconstructs a, b from RSS shares sent by $\mathcal{S}_{\text{Hon}}$ and computes $c = ab$, subsequently a random RSS of c as $\llbracket c \rrbracket_R$.

Output: $\mathcal{F}_{\text{Mul}}$ outputs corresponding RSS shares of $\llbracket c \rrbracket_R$ to all servers.

Figure 2: Ideal functionality $\mathcal{F}_{\text{Mul}}$ - RSS Multiplication with abort.

Maurer [46] in Protocol 6.2 provides maliciously secure multiplication in a similar fashion. They check for malicious behaviour by comparing the received shares and then opening the difference of all shares to confirm that parties indeed computed secret shares of the same product term $a_{T_1} b_{T_2}$. However, this approach takes three communication rounds and is communication costly.

We opt for a more efficient approach where the parties are assumed to hold a pre-shared DRSS random seed $\llbracket s \rrbracket_D$, and they use their seed share $s_{T_1 T_2}$ to compute the RSS sharing of each product term $\llbracket a_{T_1} b_{T_2} \rrbracket_R$. This way the parties can check for malicious behaviour without the need for additional communication. While this approach reduces the security to that of an underlying PRF function used for randomness generation, this trade-off is fine as the remainder of our protocol also relies on computational security (as opposed to information-theoretic in [46]).

Specifically, assume that the servers hold a common seed distributed as a DRSS. In other words let $\llbracket s \rrbracket_D = (s_{T_1 T_2})_{T_1, T_2 \in \mathcal{T}}$ be held among the servers. Server i computes

$$c^{T_1 T_2} := a_{T_1} b_{T_2},$$

for each $T_1, T_2 \in \mathcal{T}_i$, and then computes a secret sharing of c_i as follows:

$$(c_{\hat{\mathcal{T}}}^{T_1 T_2})_{\hat{\mathcal{T}} \in \hat{\mathcal{T}}} \leftarrow \text{RSS}(c^{T_1 T_2}; s_{T_1 T_2}).$$

Each server distributes the RSS shares they computed to all other servers. Upon receiving all the shares, the servers verify that the shares received from different parties corresponding to the same shares T_1, T_2, T are all equal, if not they abort, otherwise they compute the RSS of the product as

$$(ab)_T = \sum_{j=1}^n c_{j,T}.$$

Our full protocol and associated proof are presented in Appendix A.3 in a malicious adversarial setting with $t < n/3$.

4.4. RSS Multiplication with Opening

The second multiplication in Fig. 1 is computed in the online phase, where the servers compute $\llbracket x+k \rrbracket_R \llbracket s^2 \rrbracket_R$ and the client is expecting a result $(x+k)s^2$ revealed to him. We define the ideal functionality of multiplication with opening in Fig. 3 as $\mathcal{F}_{\text{MulOpen}}$, in which there are n servers $\{S_1, \dots, S_n\}$ perform multiply-and-open to a receiver $S_j, j \in \{1, \dots, n\}$. A naive way realizing $\mathcal{F}_{\text{MulOpen}}$ is to first invoke $\mathcal{F}_{\text{Mul}}$ which output $\llbracket (x+k)s^2 \rrbracket_R$, and then reveal it to receiver S_j . However, we emphasize that this is communication inefficient (large communication volume in 2 rounds). Instead, we propose a new protocol 2 that compute $\mathcal{F}_{\text{MulOpen}}$ in one round with fewer communication volumes.

In protocol 2, servers invoke $\mathcal{F}_{\text{GenZero}}$ to obtain a DRSS share of zero $\llbracket z \rrbracket_D = (z_{T_1 T_2})_{T_1, T_2 \in \mathcal{T}}$ to mask later computed local products, additionally. Note this step can also be pre-computed in the setup phase for better online efficiency. With that, for $i \in \{1, \dots, n\}$ each server i could locally compute the DRSS terms

$$o_{T_1 T_2} := a_{T_1} b_{T_2} + z_{T_1 T_2}, (T_1, T_2 \in \mathcal{T}_i)$$

for every $T_1, T_2 \in \mathcal{T}_i$. All these local terms from all servers in total constitute a random DRSS sharing of product ab , and they can be returned directly to receiver S_j . Upon receiving messages from all servers, the receiver S_j compares the consistency of $o_{T_1 T_2}^i$ for each $i \in S_{T_1 T_2}, T_1 T_2 \in \mathcal{T}$. Only if all verification pass that S_j accepts the result $v = \sum_{T_1, T_2 \in \mathcal{T}} o_{T_1 T_2}$, otherwise it rejects.

Efficiency. Each server transmits $\binom{n-1}{t}^2$ elements to the receiver S_j . Let $D = \binom{n-1}{t}^2$ denote the number of Doubly Replicated Secret Sharing (DRSS) shares of zero in Π_{dOPRF} . Note that for one party holding $\binom{n-1}{t}^2$ DRSS shares, generating each DRSS share requires $D - 1$ PRG calls. Therefore, generating DRSS shares of zero requires a total of $n \binom{n-1}{t}^2 (D - 1)$ PRG calls.

Security. Informally, we demonstrate how protocol 2 ensures both *verifiability* and *privacy*. $\mathcal{S}_{\text{Hon}}$ has access to all RSS shares of a and b , enabling them to generate DRSS shares of ab for every share index pair $T_1, T_2 \in \mathcal{T}$. This redundancy allows for the detection of any additive attack from $\mathcal{S}_{\text{Mal}}$ (verifiability).

We now address the privacy of (a, b) with respect to the receiver S_j , assuming $S_j \in \mathcal{S}_{\text{Mal}}$. Assume in the worst case that $\mathcal{A}$ controls t servers, denote $\hat{s} := \{\alpha, \beta, \dots\}$ as the set contains all indices of servers from $\mathcal{S}_{\text{Mal}}$, note adversary $\mathcal{A}$ is able to learn all RSS shares of a or b except

for the share index $\hat{T} := \hat{s}$. Thus, $\mathcal{A}$ learns nothing about DRSS shares of ab with those share indices in set

$$\hat{\mathcal{T}} = \{\hat{T}T_1, \dots, \hat{T}T_{\max}\} \cup \{T_1\hat{T}, \dots, T_{\max}\hat{T}\}.$$

In another word, $\hat{\mathcal{T}}$ of size $2\binom{n}{t} - 1$ contains indices for which the corresponding DRSS terms that are known only to $\mathcal{S}_{\text{Hon}}$. Each DRSS share with index in $\hat{\mathcal{T}}$ is masked by a random number that is known only to $\mathcal{S}_{\text{Hon}}$, thus, $\mathcal{A}$ learns nothing about each individual product $a_{T_1} b_{T_2}$ for every $T_1, T_2 \in \hat{\mathcal{T}}$. Consequently, there is no leakage (*privacy*) in Protocol 2.

Players: $S_1, \dots, S_n$ and the functionality $\mathcal{F}_{\text{MulOpen}}$.
Input: $\mathcal{S}_{\text{Hon}}$ inputs their shares $(a_T, b_T)_{T \in \mathcal{T}_i}$ to $\mathcal{F}_{\text{MulOpen}}$, $\mathcal{S}_{\text{Mal}}$ gives input $d \in \mathbb{F}_p$ to $\mathcal{F}_{\text{MulOpen}}$.
Computation: $\mathcal{F}_{\text{MulOpen}}$ reconstructs a, b from received RSS shares of $\mathcal{S}_{\text{Hon}}$, computes $c = ab + d$.
Output: If $d = 0$, $\mathcal{F}_{\text{MulOpen}}$ outputs (c, accept) to S_j . Otherwise, it outputs (c, reject) to S_j .

Figure 3: Ideal functionality $\mathcal{F}_{\text{MulOpen}}$ - RSS Multiplication with opening up to additive attack.

Protocol 2 Verifiable RSS multiplication with opening in the $\mathcal{F}_{\text{GenZero}}$ -hybrid model

Players: Server $S_1, \dots, S_n$.
Input : $\llbracket a \rrbracket_R, \llbracket b \rrbracket_R$
Output: (v, accept) or (v, reject) .

- 1 Servers invoke $\mathcal{F}_{\text{GenZero}}$ to obtain $\llbracket z \rrbracket_D$.
 - 2 **for** each server from $S_1, \dots, S_n$ **do**
 - 3 **for** $T_1, T_2 \in \mathcal{T}_i$ **do**
 - 4 Compute $o_{T_1 T_2}^i = a_{T_1}^i b_{T_2}^i + z_{T_1 T_2}$.
 - 5 **return** $\{o_{T_1 T_2}^i\}_{T_1, T_2 \in \mathcal{T}_i}$ to receiver S_R .
 - 6 S_j proceeds once it receives all messages.
 - 7 **for** $T_1, T_2 \in \mathcal{T}$ **do**
 - 8 **for** $i \in S_{T_1 T_2}$ **do**
 - 9 **if** $o_{T_1 T_2}$ is not set **then**
 - 10 Set $o_{T_1 T_2} = o_{T_1 T_2}^i$.
 - 11 **else**
 - 12 **if** $o_{T_1 T_2}^i \neq o_{T_1 T_2}$ **then**
 - 13 Outputs (v, reject) and aborts.
 - 14 Outputs $(v = \sum_{T_1, T_2 \in \mathcal{T}} o_{T_1 T_2}, \text{accept})$.
-

Theorem 2. Protocol 2 securely computes $\mathcal{F}_{\text{MulOpen}}$ in the $\mathcal{F}_{\text{GenZero}}$ -hybrid model in the presence of malicious adversaries who control $t' \leq t < n/3$ servers.

Proof. Assume $\mathcal{A}$ controls $t' \leq t$ servers, we need to prove that there exists a PPT simulator Sim that input what received by receiver S_j in $\mathcal{F}_{\text{MulOpen}}$ as well as inputs from $\mathcal{A}$, such that it's able to output a transcript that is indistinguishable to the transcript send by $\mathcal{S}_{\text{Hon}}$ in protocol 2.

Recall in ideal functionality $\mathcal{F}_{\text{MulOpen}}$, it inputs d from $\mathcal{A}$, depending on if $d = 0$ it outputs either (c, accept) or (c, reject) to the receiver S_j . While in the real world

protocol, receiver S_j receives from each server a list of messages, in which the value $o_{T_1 T_2}^i$ held by each server $i \in \mathcal{S}_i \in \mathcal{S}_{T_1 T_2}$ are supposed to be identical. Here, to argue about our claimed security, it's sufficient to simulate only those messages sent by $\mathcal{S}_{\text{Hon}}$ in the real world, from the ideal world output (c, status) and whatever the input from $\mathcal{S}_{\text{Mal}}$.

Specifically, the goal is to simulate DRSS shares sent from $\mathcal{S}_{\text{Hon}}$ to the receiver S_j . For each $T_1 T_2 \in \mathcal{T}^2 \setminus \hat{\mathcal{T}}$, since $\mathcal{A}$ has all the knowledge of RSS share of a, b with RSS indices in $\mathcal{T} \setminus \{\alpha, \beta, \dots\}$, $\mathcal{A}$ could identically generate all DRSS shares with indices in $\mathcal{T}^2 \setminus \hat{\mathcal{T}}$, now the only task remained is to simulate DRSS share indices in $\hat{\mathcal{T}}$. Continually, let Sim samples $2\binom{n}{t} - 2$ random numbers from $\mathbb{F}_p$, and compute

$$\bar{o} = c - d - \sum_{T_1, T_2 \in \mathcal{T}^2 \setminus \hat{\mathcal{T}}} o_{T_1 T_2}^i - \bar{r}$$

where $\bar{r}$ denotes the sum of these random numbers. Putting these $2\binom{n}{t} - 2$ numbers and $\bar{o}$ altogether, we are able to obtain a list $\{\{\hat{o}_{T_1 T_2}^i\}_{T_1, T_2 \in \mathcal{T}_i}\}_{i \in \mathcal{S}_{\text{Hon}}}$ of size $2\binom{n}{t} - 1$, which constitute the simulated transcript for the DRSS shares sent by $\mathcal{S}_{\text{Hon}}$ in protocol 2.

Thus, $\mathcal{A}$ perfectly simulate transcripts sent from $\mathcal{S}_{\text{Hon}}$. $\square$

5. The Legendre dOPRF protocol

In the framework of Fig. 1, if a malicious client colludes with $t' \leq t < n/3$ servers, protocol 2 is not applicable because we cannot guarantee that the input RSS of x is valid; that is, each server in $\mathcal{S}_{\text{Hon}}$ must receive identical RSS shares corresponding to the same RSS index.

For this reason, we first define the desired ideal functionality $\mathcal{F}_{\text{dOPRF}}$ in Fig. 4 and then present our adapted protocol 3 that securely computes $\mathcal{F}_{\text{dOPRF}}$ against an adversary $\mathcal{A}$ that controls both the client and $t' \leq t < n/3$ servers.

5.1. Ideal functionality in the malicious setting

We assume that an adversary $\mathcal{A}$ corrupts $t' \leq t < n/3$ servers and the client in Fig. 1, and $\mathcal{A}$ might act arbitrarily the way he desires. Specifically,

- $\mathcal{A}$ may corrupt the client to submit inconsistent RSS of x to honest servers to extract k ;
- $\mathcal{A}$ may corrupt those t' servers to introduce additive attack when computing multiply-and-open among servers.

Thus, in Fig. 4 we are able to define the desired ideal functionality $\mathcal{F}_{\text{dOPRF}}$ in the $(\mathcal{F}_{\text{GenZero}}, \mathcal{F}_{\text{GenRand}})$ -hybrid model. $\mathcal{F}_{\text{dOPRF}}$ interacts with the client and all n servers in the online phase. It serves as an ideal functionality that inputs $x \in \mathbb{F}_p$ or $\perp$ from the client, honest RSS shares of k, s^2 from $\mathcal{S}_{\text{Hon}}$ and z from $\mathcal{A}$, and outputs to the client a computation result $(\alpha \in \{r \leftarrow \mathbb{F}_p, (x+k)s^2+d\}, \text{status} \in \{\text{accept}, \text{reject}\})$.

With the definition of the ideal functionality in Fig. 4, it is clear that when $\mathcal{A}$ inputs $\perp$ (modeling inconsistent RSS in real world) to $\mathcal{F}_{\text{dOPRF}}$, the adversary learns only

a random element $r \in \mathbb{F}_p$ after computation, thus gains no information of k or s^2 . On the other hand, for $\mathcal{A}$ that controls t' servers, he cannot force an honest client to accept a computation result $c = (x+k)s^2+d$ when $d \neq 0$.

Players: Server S_i ; $i = 1, \dots, n$, the client and $\mathcal{F}_{\text{dOPRF}}$.

Input: Client inputs $x \in \mathbb{F}_p$ or $\perp$ to $\mathcal{F}_{\text{dOPRF}}$. Denotes s^2 as b , $\mathcal{S}_{\text{Hon}}$ inputs their shares $(k_T, b_T)_{T \in \mathcal{T}_i}$ to $\mathcal{F}_{\text{dOPRF}}$, $\mathcal{A}$ gives input $d \in \mathbb{F}_p$ to $\mathcal{F}_{\text{dOPRF}}$.

Computation: If the client inputs $x \in \mathbb{F}_p$, $\mathcal{F}_{\text{dOPRF}}$ reconstructs k, b from received RSS shares of $\mathcal{S}_{\text{Hon}}$, computes $c = (x+k)b + d$.

Output: If $x \in \mathbb{F}_p$, when $d = 0$, $\mathcal{F}_{\text{dOPRF}}$ outputs (c, accept) to client and (c, reject) otherwise. When $x = \perp$, set $c = r$ where r is uniformly randomly selected from $\mathbb{F}_p$, $\mathcal{F}_{\text{dOPRF}}$ outputs (r, reject) to the client.

Figure 4: Ideal functionality $\mathcal{F}_{\text{dOPRF}}$ up to additive attack.

5.2. Malicious Legendre dOPRF protocol

A straightforward approach to implementing $\mathcal{F}_{\text{dOPRF}}$ involves first verifying the consistency of the client's inputs among $\mathcal{S}_{\text{Hon}}$. This verification process requires each participating server to confirm with every other server that holds the same RSS share. If any inconsistent RSS share of x is detected, the servers collectively output $[r]$, generated by $\mathcal{F}_{\text{GenRand}}$, and then issue a reject signal. Otherwise, $\mathcal{F}_{\text{MulOpen}}$ is invoked to compute $(x+k)s^2$. However, even realizing $\mathcal{F}_{\text{MulOpen}}$ with our communication optimized protocol 2, this method of verifying client inputs leads to still three communication rounds.

In this subsection, we adapt protocol 2 to protocol 3 that realize $\mathcal{F}_{\text{dOPRF}}$ in two rounds, which is also denoted as Π_{dOPRF} throughout. To check potential inconsistent RSS of x submitted to $\mathcal{S}_{\text{Hon}}$, we introduce a small change in protocol 3: for every $T_1 T_2 \in \mathcal{T}$, we have each server $i \in \mathcal{S}_{T_1 T_2}$ send $v_{T_1 T_2}^i = o_{T_1 T_2}^i + (x_{T_1}^i + r_{T_1}^i)\hat{z}_{T_1 T_2}^i$ to the client instead of $o_{T_1 T_2}^i$ as in protocol 2, where $[r]$ is a random ASS among all servers and $\hat{z}_{T_1 T_2}$ is an ASS shared among $\mathcal{S}_{T_1 T_2}$. With this change, only if all honest parties receive consistent shares of x can the client reconstruct $o_{T_1 T_2}$. Otherwise, the client receives random messages and learns nothing of k or s^2 . In other words, if the client acts maliciously and sends an invalid RSS of x to $\mathcal{S}_{\text{Hon}}$, the client will receive a random output, preventing the adversary from deducing any useful information. However, above change in protocol 3 breaks the verifiability by an honest client against additive attack. To remedy this loss, we have each server send additionally one single hash output h_i computed over $\{o_{T_1 T_2}^i\}_{T_1, T_2 \in \mathcal{T}_i}$ to the client. After an honest client reconstructs $\{o_{T_1 T_2}\}_{T_1, T_2 \in \mathcal{T}}$, for each $i \in [n]$ he recomputes his own hash output over $\{o_{T_1 T_2}\}_{T_1, T_2 \in \mathcal{T}_i}$ and compares it to h_i , he accepts $\{o_{T_1 T_2}\}_{T_1, T_2 \in \mathcal{T}}$ only if all checks pass. Here, altogether servers send $n \left(\binom{n-1}{t}^2 + 1 \right)$ field elements to the client.

Protocol 3 Legendre dOPRF in the $(\mathcal{F}_{\text{GenZero}}, \mathcal{F}_{\text{GenRand}})$ -hybrid model (Π_{dOPRF})

Players: Server S_i for $i = 1, \dots, n$ and a client.

Input : $x \in \mathbb{F}_p$ by client, $\llbracket s^2 \rrbracket_R$ and $\llbracket k \rrbracket_R$ by servers.

Output: (c, status) .

- 1 Client computes a RSS of $x = \sum_{T \in \mathcal{T}} x_T$, then sends $(x_T)_{T \in \mathcal{T}_i}$ to server S_i for each $i \in \{1, \dots, n\}$.
 - 2 Servers invoke $\mathcal{F}_{\text{GenZero}}$ to obtain $\llbracket z \rrbracket_D$, $[\hat{z}_{T_1 T_2}]$ among $\mathcal{S}_{T_1 T_2}$ for each $T_1, T_2 \in \mathcal{T}$; additionally, they invoke $\mathcal{F}_{\text{GenRand}}$ to obtain $\llbracket r \rrbracket_R$.
 - 3 Let $\llbracket a \rrbracket_R = \llbracket x + k \rrbracket_R$, $\llbracket b \rrbracket_R = \llbracket s^2 \rrbracket_R$.
 - 4 **for** each server i from $S_1, \dots, S_n$ **do**
 - 5 Set $H = \epsilon$.
 - 6 **for** $T_1, T_2 \in \mathcal{T}_i$ **do**
 - 7 Compute $o_{T_1 T_2}^i = a_{T_1}^i b_{T_2}^i + z_{T_1 T_2}$.
 - 8 Update $H = H \parallel o_{T_1 T_2}^i$.
 - 9 Compute $v_{T_1 T_2}^i = o_{T_1 T_2}^i + (x_{T_1}^i + r_{T_1}^i) z_{T_1 T_2}^i$.
 - 10 Compute $h_i = \text{Hash}(H)$.
 - 11 **return** $(\{v_{T_1 T_2}^i\}_{T_1, T_2 \in \mathcal{T}_i}, h_i)$ to client.
 - 12 Once the client receives all messages, it proceeds as follows.
 - 13 Set $v = 0$.
 - 14 **for** $T_1, T_2 \in \mathcal{T}$ **do**
 - 15 Computes $v_{T_1, T_2} = \frac{1}{\#\mathcal{S}_{T_1, T_2}} \sum_{i \in \mathcal{S}_{T_1, T_2}} v_{T_1, T_2}^i$.
 - 16 Accumulates $v = v + v_{T_1, T_2}$.
 - 17 **for** $i = 1, \dots, n$ **do**
 - 18 Set $H = \epsilon$.
 - 19 **for** $T_1, T_2 \in \mathcal{T}_i$ **do**
 - 20 Update $H = H \parallel v_{T_1 T_2}$.
 - 21 If $h_i \neq \text{Hash}(H)$, outputs (v, reject) .
 - 22 Outputs (v, accept) .
-

5.2.1. Client Verifiability. Consider a scenario where an honest client interacts with up to t servers that may be controlled by an adversary $\mathcal{A}$. In this setting, we assert the verification algorithm described in the reconstruction stage of protocol 3 is an efficient verification algorithm such that:

- If client inputs inconsistent RSS shares of x to $\mathcal{S}_{\text{Hon}}$, client receives (r, reject) ;
- Otherwise, if $\mathcal{S}_{\text{Mal}}$ inputs $d \in \mathbb{F}_p$ in the additive attack, client receives

$$c, \text{status} = \begin{cases} (x + k)s^2 + d, \text{reject} & \text{if } d \neq 0, \\ (x + k)s^2, \text{accept} & \text{otherwise.} \end{cases}$$

This verifiability ensures that an honest client can reliably verify whether the protocol execution was free from disruptions, in the presence of up to t corrupted servers.

Theorem 3. *Protocol 3 securely computes $\mathcal{F}_{\text{dOPRF}}$ in the $(\mathcal{F}_{\text{GenZero}}, \mathcal{F}_{\text{GenRand}})$ -hybrid model in the presence of malicious adversaries who control $t' \leq t < n/3$ servers and the client.*

Proof. We need to prove there exists a PPT simulator Sim, it inputs only the received output (c, status) by the client in the ideal functionality $\mathcal{F}_{\text{dOPRF}}$, such that it's able to

output a transcript whose distribution is indistinguishable to the one received by the client in protocol 3. Specifically, the goal is to simulate $v_{T_1 T_2}^i$ for every $i \in \mathcal{S}_{\text{Hon}}, T_1, T_2 \in \hat{\mathcal{T}}$ and $\{h_i\}_{i \in \mathcal{S}_{\text{Hon}}}$. *w.l.o.g.*, assume $t' = t$, which indicates $|\mathcal{S}_{\text{Hon}}| = n - t$, since each honest server holds RSS share index $\hat{T}$ and thus he holds $2\binom{t}{n-1}$ terms whose DRSS share indices are within $\hat{\mathcal{T}}$. Thus, there are in total $I = (n - t)(2\binom{t}{n-1})$ terms sent from $\mathcal{S}_{\text{Hon}}$ corresponds to $\hat{\mathcal{T}}$.

In the case when the client submits consistent RSS of x to $\mathcal{S}_{\text{Hon}}$, to simulate their distribution in this case, let Sim draws randomly field elements $\{r_i\}_{i \in [I-1]}$ from $\mathbb{F}_p$, where $\bar{r}$ is denoted as their sum. Denote

$$\bar{v} = c - d - \sum_{i \in [I-1]} r_i - \bar{r},$$

the list $L = \{r_1, \dots, r_{I-1}, \bar{v}\}$ of size I is exactly the generated simulation of $v_{T_1 T_2}^i$ for every $i \in \mathcal{S}_{\text{Hon}}, T_1, T_2 \in \hat{\mathcal{T}}$, as the distribution of L is identical to what the client receives in the real world. Then, based from simulated list L , the simulator is able to compute $o_{T_1 T_2}$ for every $T_1 T_2 \in \hat{\mathcal{T}}$ and hence perfectly computes $\hat{h}_i$ for every $i \in \mathcal{S}_{\text{Hon}}$.

In contrast, when the client provides inconsistent RSS share of x to $\mathcal{S}_{\text{Hon}}$, we analyze the message distribution in the real world. *w.l.o.g.*, we assume that the client submits a *valid* $\llbracket x \rrbracket_R$ except at share index T_* . This means there are inconsistent values for the share x_{T_*} held by $\mathcal{S}_{T_*}$. Formally, we define $\tilde{x}_{T_*}^i = e^i + x_{T_*}$, where there exist $i, j \in \mathcal{S}_{T_*}$ such that $e^i \neq e^j$. Here, e^i denotes the error in $\tilde{x}_{T_*}^i$. Thus, $\tilde{x}_{T_*}^i$ represents a potentially invalid share held by server S_i with share index T_* . Denote $\bar{\mathcal{T}} = \{T_* T_1, \dots, T_* T_{\max}\}$, for every $T_1 T_2 \in \bar{\mathcal{T}}$, observe that in protocol 3 the client reconstructs

$$v_{T_1 T_2} = \frac{1}{\#\mathcal{S}_{T_1 T_2}} \left(\sum_{i \in \mathcal{S}_{T_1 T_2}} a_{T_1}^i b_{T_2}^i \right) + e_{T_1 T_2}$$

where $e_{T_1 T_2} = \frac{1}{\#\mathcal{S}_{T_1 T_2}} \sum_{i \in \mathcal{S}_{T_1 T_2}} e_{T_1}^i z_{T_1 T_2}^i$ is a linear combination result from random numbers that obeys a uniform distribution $\mathcal{U}$, thus the final reconstructed output $v_{T_1 T_2}$ is independent of secret a or b . Finally, the client is able to recover a value $c = V + E$, where V is related to k and s^2 and $E = \sum_{T_1 T_2 \in \bar{\mathcal{T}}} e_{T_1 T_2}$ is a random number.

From above analysis, we present the simulator construction in the following, where the simulated view is indistinguishable to that in the execution of protocol 3. To simulate the transcript in the real world from $\mathcal{F}_{\text{dOPRF}}$ output (r, reject) :

- Let Sim draw randomly I field elements from $\mathbb{F}_p$ as simulated view transcript for $v_{T_1 T_2}^i, T_1 T_2 \in \hat{\mathcal{T}}, i \in \mathcal{S}_{\text{Hon}}$.
- For each $i \in \mathcal{S}_{\text{Hon}}$, Let Sim draw a random element $\hat{h}_i$ from the output domain of Hash as simulated transcript of $\hat{h}_i$.

Finally, Sim is able to output:

$$\{\{\hat{v}_i^{T_1 T_2}\}_{T_1, T_2 \in \mathcal{T}_i}, \hat{h}_i\}_{i \in \mathcal{S}_{\text{Hon}}}.$$

In the random oracle model, if we draw a random element $\hat{h}_i$ from the output domain of Hash, we argue

that any PPT $\mathcal{A}$ cannot distinguish between $\hat{h}_i$ and the real execution output h_i .

Thus, $\mathcal{A}$ simulates all transcripts sent from S_{Hon} that is computationally indistinguishable to what he receives in the real world. $\square$

5.3. Full security of the Legendre PRF

Theorem 3 establish that securely evaluating a single bit of the Legendre PRF in an oblivious manner is guaranteed, assuming up to $t < n/3$ malicious servers. This ensures that either the protocol runs honestly, or if there is a malicious party, no one gains any information about the secret values. As a result, the oblivious Legendre PRF effectively simulates an idealized Legendre PRF oracle $\mathcal{F}_{\text{dOPRF}}$. The security of the PRF, where an attacker has access to the Legendre PRF oracle $\mathcal{F}_{\text{dOPRF}}$, is based on Assumption 4. The most effective known attacks against Assumption 4 are key-extraction attacks [30], [41], [42]. Given that an attacker can query the PRF M times, the classical attack requires $O(M^2 \log p)$ memory and has a runtime of $O(\frac{p \log p \log \log p}{M^2})$, while the quantum attack uses $\tilde{O}(M^2)$ classical memory with quantum random access and has a runtime of $\tilde{O}(\sqrt{\frac{p}{M^2}})$. Under these conditions, the protocol offers $\lambda_c = \lambda$ bits of classical security and $\lambda_q = 2\lambda/3$ bits of quantum security when the underlying prime p is of size 2λ .

5.4. On partial obliviousness

A partially oblivious PRF (pOPRF) is an OPRF whose input is divided into a private and a public part $x = (x_{\text{pub}}, x_{\text{priv}})$. There are generic transformations ([13], Section 3.1) to construct a pOPRF from an OPRF, which can also be employed in our construction. Let $\{F_k\}$ be an OPRF, let $\{G_k\}$ be a PRF. Then, the function:

$$F'_k(x_{\text{pub}}, x_{\text{priv}}) := F_{G_k(x_{\text{pub}})}(x_{\text{priv}}),$$

is a pOPRF. Given a fixed RSS secret sharing of a key $k = \sum_{T \in \mathcal{T}} k_T$, we define:

$$k' = \sum_{T \in \mathcal{T}} G_{k_T}(x_{\text{pub}})$$

to be an updated OPRF key, which we can also denote as $G'(k) = k'$. Here G' is a PRF function, which is not only a function on k , but on the randomness involved in splitting k into RSS shares. The new key k' can be computed from the public PRF G , the local shares k_T and the public part of the input x_{pub} . The partially oblivious Legendre PRF is defined to be the Legendre OPRF computed with the updated key k' , both in the semi-honest and malicious setting.

The main drawback of this approach is that it depends on the secret sharing of the long-term key k . A different secret sharing leads to a different function which makes the protocol insecure against adaptive adversaries.

5.5. On adaptive adversaries

An adaptive adversary is an adversarial model in which the set of corrupted servers can change over time.

However, the adversary stays constrained to control at most t corrupted servers at a time.

Our protocol can be made adaptively secure by updating all long term secret data at regular time intervals. These include the key $\llbracket k \rrbracket_R$, as well as pre-shared strings of random shares discussed in Appendix A. The parties can agree on a common random value by means of any protocol, and then compute a random secret sharing of zero by employing the protocol of Appendix A.2 and update the key and any shared seeds. Updating the key, however, breaks the partial obliviousness of the protocol.

6. Comparison to Existing MPC Solutions

We compare our protocol Π_{dOPRF} to two state-of-the-art MPC solutions proposed by Dalskov et al. [18] and Chida et al. [16] and present our results in Table 2. Specifically, we consider the same but practical outsourced computation model with one client and a few servers, where one or two servers are corrupted. Note that we adapt the computation of $\mathcal{C}_{\text{PRF}}$ to fit our desired outsourced computation model for works in [16], [18].

In the work of Dalskov et al. [18], they consider a setting of four parties where one party might be an active adversary, the authors propose a novel method performing multiplication as well as identifying cheating. This method incurs a total of 12 elements exchanged for each multiplication operation. Including the communication cost for the input and output stages, the total communication volume amounts to 36 elements over 3 rounds.

In the work of Chida et al. [16], the authors propose a general MPC framework designed for settings with an honest majority in the presence of $t < n/2$ malicious adversaries. This allows more room for adversarial behaviours than our work, which allows less corruptions with $t < n/3$. Their framework is modular and applicable to any arbitrary arithmetic circuit, and it consists four stages: *Input Sharing*, *Circuit Emulation*, *Verification*, and *Output Reconstruction*. However, their solution is not as efficient when applied to our proposed dOPRF. More precisely, for computing the circuit $\mathcal{C}_{\text{PRF}}$, regardless of the underlying secret sharing scheme used, the framework requires 7 invocations of $\mathcal{F}_{\text{rand}}$, 6 invocations of $\mathcal{F}_{\text{mul}}$, and 8 invocations of $\mathcal{F}_{\text{open}}$ over 9 rounds. If using RSS- (t, n) where $(t, n) = (1, 3)$, assuming the underlying semi-honest sub-protocol $\mathcal{F}_{\text{mul}}$ is instantiated by a naive protocol consuming 24 elements in one round, $\mathcal{F}_{\text{open}}$ instantiated with 6 elements, this results in a total communication volume of 201 elements. When $(t, n) = (2, 5)$, assuming the underlying semi-honest sub-protocol $\mathcal{F}_{\text{mul}}$ is instantiated by the protocol in [20], which requires opening two masked elements among all parties, the communication volume would total 745 elements. Alternatively, if using Shamir Secret Sharing in the setting $(t, n) = (2, 5)$, communication-wise, the realization of $\mathcal{F}_{\text{mul}}$ requires 20 elements and $\mathcal{F}_{\text{open}}$ also requires 20 elements, resulting a total communication of 445 elements.

In Tab. 2, with one corrupt server, we compared our protocol to the solution built from Fantastic Four [18] that used both RSS- $(1, 4)$ and the solution from [16] built from RSS- $(1, 3)$; with two corrupt servers, we compared our protocol instantiated from RSS- $(2, 7)$ to the solution from [16] built from two secret sharing schemes, *i.e.*

TABLE 2: Concrete efficiency comparison between Π_{dOPRF} and state-of-the-art MPC solutions, with 1 or 2 corrupt servers.

Reference	Scheme	Comm.	No Inter-Server Comm.	Rounds
Our Π_{dOPRF}	RSS-(1,4)	52	✓	2
Dalskov19 [18]	RSS-(1,4)	36	✗	3
Chida23 [16]	RSS-(1,3)	201	✗	9
Our Π_{dOPRF}	RSS-(2,7)	1687	✓	2
Chida23 [16]	RSS-(2,5)	745	✗	9
Chida23 [16]	SSS-(2,5)	445	✗	9

RSS-(2,5) and SSS-(2,5). As clearly shown in Tab. 2, only our work Π_{dOPRF} offers the advantage of no inter-server communication. In the setting of one corrupt server, since computation and communication are less dominant than communication delay, our protocol Π_{dOPRF} exhibits a clear advantage over the state-of-the-art. With two corrupt servers, our protocol Π_{dOPRF} requires roughly double or triple the communication volume than in [16], regardless of this higher volume cost, we argue with much fewer rounds required in our work: 2 rounds compared to 9 rounds, our protocol Π_{dOPRF} would be more efficient than [16] in this application scenario.

7. Evaluation

We implemented and tested Π_{dOPRF} in C language and compiled using gcc with the `-O3` optimization level. We use the BLAKE3 [49] as a hash function and also as a PRF for generating randomness. We performed our tests on a single server with 32GB RAM, 12th Gen Intel(R) Core(TM) i7-12700K CPU model that ran Ubuntu 22.04 LTS.

Our code is available on GitHub, we provide two versions: one for local computation benchmarking¹ and another adapted for WAN testing to compare to state-of-the-art OPRF solutions².

7.1. Detailed benchmarks

We provide detailed benchmarks of run-time and communication volume of each subroutine in the protocol in Tab. 3 and 4.

The `setup_zeros` refers to the local computation of secret shares with a PRF as explained in A.2. To that end we use the BLAKE3 function as a PRF, with seed size of the same size as the prime. This part of the algorithm scales the worst due to the large size of the seeds used. In the malicious setting, the seed for generating DRSS zeros is of size $\approx \#\mathcal{T}_i^2 \times \mathcal{T}^2 = \binom{n-1}{t}^2 \binom{n}{t}^2$ which we can crudely approximate with n^{4t} , which is quadruply exponential for $t \propto n$.

The `setup_mul` is the multiplication of the square $\llbracket s^2 \rrbracket_R$ in the setup phase which takes one round of communication. The run-times for the computation in the two rounds is presented separately. Again, the bottleneck in the malicious case is the size of the messages sent which are $\#\mathcal{T}_i^2 \times \mathcal{T} \approx \binom{n-1}{t}^2 \binom{n}{t}$ field elements.

1. <https://github.com/nkKolja/Legendre-dOPRF>

2. <https://github.com/nann-cheng/d-OPRF>

The input, evaluation, and reconstruction sub-procedures are as presented in protocol 3. The Legendre symbol computation is a part of the reconstruction algorithm. However, it is benchmarked separately as `leg_symbols`.

Finally, with λ we denote the number of output bits, and with λ_c and λ_q the classical and quantum security parameters.

We note that our algorithm is highly parallelisable, and we did not exploit that property in our implementation. Up to a λ factor speedup can be expected with parallelisation.

Communication volume wise, in total there are two rounds of communication when data is sent. The first one in the setup phase during the computation of the random square, and the second one when the client communicates with the servers and receives the output.

In the setup phase each server sends $\lambda \times \#\mathcal{T}_i^2$ many RSS shares to each party, amounting to $\binom{n-1}{t}^3 \lambda^2/4$ bytes per message. Each server receives a message from each other server totalling $n \binom{n-1}{t}^3 \lambda^2/4$ bytes. In the online phase, the client sends one RSS share - $\binom{n-1}{t}^3 \lambda/4$ bytes to each server. He receives $\lambda \times \#\mathcal{T}_i^2$ DRSS shares from each server, along with a single hash digest amounting to $\lambda \times \binom{n-1}{t}^2 \times \lambda/4 + \lambda/4$ bytes per server message. We note that in our implementation we rehash all λ digests to a single digest. This allows for faster computation and verification of the online phase without compromising security.

7.2. Comparison with state-of-the-art OPRF

All protocol evaluations were conducted within the same simulated WAN setting on a single machine, where we use the `tc` command to simulate a network environment with a bandwidth of 50 Mbits and a 100 ms delay.

In Tab. 5 we compare our protocol to state-of-the-art works in terms of concrete computation time and communication volume, covering both the offline and online phase for a output length of 128 bits. Those state-of-the-art works are from two categories: two-party post-quantum OPRF protocols [11], [25] and distributed classical OPRF protocols [10]. For each protocol evaluation, λ indicates the security level, *Comm* represents the total communication volume, *Rounds* denotes the number of sequential communication rounds in the online phase, and *Runtime* measures the evaluation time at different stages. Specifically, for the offline phase, *Runtime* corresponds to the total time required to complete all tasks by all involved parties. For the online phase, we measure the time from when the client initiates a request to the server until the final OPRF/dOPRF result is obtained. In Tab. 5, the result for [10] is an estimation from the figures in Tab. 1 of [10] (2 servers) adjusted to our WAN setting (200 ms RTT); the numbers for [11] are taken from their estimations in Section 5.2 of [11] under our WAN setting. As for GC-OPRF [25], we run their code in the simulated WAN setting and obtain corresponding numbers. Note that as for our Π_{dOPRF} , we revised our results to align with those from a real distributed server set up, based on the outcomes we achieve from a single-machine experiment, since distributed server setup is anticipated to exhibit improved performance, considering it allows for distributed computations and parallel network communications.

TABLE 3: Timing benchmarks for the Legendre dOPRF.

p	λ	λ_c	λ_q	t	n	setup zeros	setup mul	input	evaluation	reconstruction	leg_symbols
128 bits	64	64 bits	43 bits	1	4	0,2 ms	0,11 + 0,04 ms	0,0001 ms	0,04 ms	0,04 ms	0,10 ms
				2	7	121 ms	17,0 + 7,0 ms	0,001 ms	0,91 ms	1,41 ms	
				3	10	120 s	4,50 + 3,9 s	0,008 ms	29,7 ms	81,0 ms	
192 bits	96	96 bits	64 bits	1	4	0,4 ms	0,22 + 0,07 ms	0,001 ms	0,09 ms	0,091 ms	0,44 ms
				2	7	230 ms	27,0 + 13,0 ms	0,001 ms	2,17 ms	2,707 ms	
				3	10	228 s	7,6 + 6,8 s	0,008 ms	71,15 ms	148,4 ms	
256 bits	128	128 bits	86 bits	1	4	0,8 ms	0,41 + 0,09 ms	0,001 ms	0,194 ms	0,172 ms	1,34 ms
				2	7	450 ms	46,0 + 16,0 ms	0,001 ms	4,681 ms	5,383 ms	
				3	10	446 s	13,1 + 9,2 s	0,022 ms	159,0 ms	272,2 ms	

TABLE 4: Communication volume benchmarks for the Legendre dOPRF.

p	λ	t	n	setup_mul sent	setup_mul received	input sent	evaluation sent	reconstruction received
128 bits	64	1	4	27 KB	108 KB	48 B	9 KB	36 KB
		2	7	3,3 MB	23,1 MB	240 B	225 KB	1,5 MB
		3	10	592 MB	5,9 GB	1,3 KB	7 MB	70 MB
192 bits	96	1	4	61 KB	244 KB	72 B	20,5 KB	82 KB
		2	7	7,5 MB	52,5 MB	360 B	517 KB	3,6 MB
		3	10	1,3 GB	11,3 GB	1,9 KB	16 MB	160 MB
256 bits	128	1	4	108 KB	432 KB	96 B	36 KB	144 KB
		2	7	13,1 M	91,7 MB	480 B	907 KB	6,3 MB
		3	10	2,3 GB	23 GB	2,6 KB B	28,4 MB	284 MB

TABLE 5: Performance comparison between our work and state-of-the-art OPRF solutions. Superscript * indicates estimated numbers adapted to our WAN setting.

Protocol	λ	Primitive	Comm.[KB]		Rounds	Runtime [ms]	
			Offline	Online		Offline	Online
PESTO [10]	128	Pairing, ECDSA	-	-	2	-	260*
GC-OPRF [25]		GC, AES-128	-	4634	4	-	1842
Legendre-OPRF [11]		OT, ZK	-	911	9	-	1049*
Our Π_{dOPRF}		RSS-(1,4)	432	144	2	204	215
Our Π_{dOPRF}		RSS-(2,7)	93900	6303	2	1076	815
GC-OPRF [25]	256	GC, AES-256	-	6628	4	-	2320
Our Π_{dOPRF}		RSS-(1,4)	1728	579	2	208	637

Among all protocols achieving $\lambda = 128$ bits security, our protocol Π_{dOPRF} with RSS-(1,4) instantiation performed the best in terms of both computational and communication efficiency. Additionally, compared to [25], at a security level of $\lambda = 256$ our protocol Π_{dOPRF} also showed better efficiency with the parameter setting $(t, n) = (1, 4)$.

8. Future work

Our work concentrated on the online phase of the algorithm. For the setup phase we used basic RSS and correlated randomness algorithms which make it run super-exponentially in (t, n) in both communication volume and run-time. We believe that faster alternatives exist. For example there is no need that the inputs to the squaring algorithm are RSS, only the output needs to be RSS. Also, we use communicationless methods to compute secret shares of zero, but since there is already a round of communication in the setup phase, it can be leveraged to speed up this computation. We believe that speedups for $t < 4$ are possible.

Furthermore, while the communication model depicted in Fig. 1 leads to an efficient dOPRF, it contains a weakness in that compromising a threshold of the servers reveals the client's input in addition to the secret key.

Constructing a post-quantum dOPRF which leaks only the key when the adversary corrupts all the servers remains an open problem, and we are confident that our ideas will be useful towards reaching those goals.

Reconstructing the output requires honest participation of all n servers. A threshold protocol which allows for reconstruction when $< n$ servers participate is an open research direction, and using RSS for that purpose without increasing the cost too much seems non-trivial.

Finally secure multiparty computation in an honest majority scenario is used for running machine-learning algorithms in a privacy preserving manner [15], [47], [48], [52]. Both malicious multiplication protocols 2, 3 and 4 could potentially be used in those scenarios with an additional layer of verifiability on top. Finding practical use-cases for these algorithms is an interesting problem to pursue.

References

- [1] Secret sharing schemes realizing general access structures. In *Proc. IEEE Global Telecommunication Conf., Globecom '87*, Lecture Notes in Computer Science, 1987.
- [2] Unapproved IEEE Draft Standard for Specifications for Password Based Public Key Cryptographic Techniques. *IEEE Unapproved Std P1363.2 /D27*, 2007.

- [3] ISO: Information technology — security techniques — key management — part 4: Mechanisms based on weak secrets. ISO/IEC, International Organization for Standardization, 2017. <https://www.iso.org/standard/67933.html>16.
- [4] Shashank Agrawal, Peihan Miao, Payman Mohassel, and Pratyay Mukherjee. Pasta: Password-based threshold authentication. Cryptology ePrint Archive, Paper 2018/885, 2018.
- [5] Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. Crypto dark matter on the torus: Oblivious prfs from shallow prfs and fhe. Cryptology ePrint Archive, Paper 2023/232, 2023.
- [6] Martin R. Albrecht, Alex Davidson, Amit Deo, and Nigel P. Smart. Round-optimal verifiable oblivious pseudorandom functions from ideal lattices. Cryptology ePrint Archive, Paper 2019/1271, 2019.
- [7] Alessandro Baccarini, Marina Blanton, and Chen Yuan. Multi-party replicated secret sharing over a ring with applications to privacy-preserving machine learning. Cryptology ePrint Archive, Paper 2020/1577, 2020.
- [8] Andrea Basso. A post-quantum round-optimal oblivious prf from isogenies. Cryptology ePrint Archive, Paper 2023/225, 2023.
- [9] Andrea Basso, Péter Kutas, Simon-Philipp Merz, Christophe Petit, and Antonio Sanso. Cryptanalysis of an oblivious prf from supersingular isogenies. In *Advances in Cryptology – ASIACRYPT 2021*.
- [10] Carsten Baum, Tore K. Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. Pesto: Proactively secure distributed single sign-on, or how to trust a hacked server. Cryptology ePrint Archive, Paper 2019/1470, 2019.
- [11] Ward Beullens, Lucas Dodgson, Sebastian Faller, and Julia Hesse. The 2hash oprf framework and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/450, 2024.
- [12] Dan Boneh, Dmitry Kogan, and Katharine Woo. Oblivious pseudorandom functions from isogenies. Cryptology ePrint Archive, Paper 2020/1532, 2020.
- [13] Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. Sok: Oblivious pseudorandom functions. Cryptology ePrint Archive, Paper 2022/302, 2022.
- [14] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: An efficient post-quantum commutative group action. Cryptology ePrint Archive, Paper 2018/383, 2018.
- [15] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. Trident: Efficient 4pc framework for privacy preserving machine learning. Cryptology ePrint Archive, Paper 2019/1315, 2019. <https://eprint.iacr.org/2019/1315>.
- [16] Koji Chida, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Daniel Genkin, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority mpc for malicious adversaries. *Journal of Cryptology*, 36(3):15, 2023.
- [17] Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*, volume 3378 of *Lecture Notes in Computer Science*, pages 342–362. Springer, 2005.
- [18] Anders Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: {Honest-Majority} {Four-Party} secure computation with malicious security. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2183–2200, 2021.
- [19] Ivan Damgård. On the randomness of Legendre and Jacobi sequences. In *Proceedings of the 8th Annual International Cryptology Conference on Advances in Cryptology, CRYPTO '88*, pages 163–172, London, UK, UK, 1990. Springer-Verlag.
- [20] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *Annual International Cryptology Conference*, pages 572–590. Springer, 2007.
- [21] Poulami Das, Julia Hesse, and Anja Lehmann. Dpase: Distributed password-authenticated symmetric encryption. Cryptology ePrint Archive, Paper 2020/1443, 2020.
- [22] Itai Dinur, Steven Goldfeder, Tzipora Halevi, Yuval Ishai, Mahimna Kelkar, Vivek Sharma, and Greg Zaverucha. Mpc-friendly symmetric cryptography from alternating moduli: Candidates, protocols, and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*.
- [23] Lucas Dodgson, 2023. https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/appliedcrypto/education/theses/Master_Thesis_Post_Quantum_Building_blocks_for_secure_computation.pdf.
- [24] Daniel Escudero. An introduction to secret-sharing-based secure multiparty computation. Cryptology ePrint Archive, Paper 2022/062, 2022.
- [25] Sebastian Faller, Astrid Ottenhues, and Johannes Ottenhues. Composable oblivious pseudo-random functions via garbled circuits. Cryptology ePrint Archive, Paper 2023/1176, 2023.
- [26] Dankard Feist. Legendre pseudo-random function, 2019. <https://legendreprf.org/bounties>.
- [27] Luca De Feo, David Jao, and Jérôme Plût. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. Cryptology ePrint Archive, Paper 2011/506, 2011.
- [28] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *Theory of Cryptography*, pages 303–324, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [29] Eduarda S. V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. Non-interactive key exchange. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *Public-Key Cryptography – PKC 2013*, pages 254–271, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [30] Paul Frixons and André Schrottenloher. Quantum security of the legendre prf. Cryptology ePrint Archive, Paper 2021/149, 2021.
- [31] Niv Gilboa and Yuval Ishai. Compressing cryptographic resources. In *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 591–608. Springer, 1999.
- [32] Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. MPC-friendly symmetric key primitives. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16*, pages 430–443, New York, NY, USA, 2016. ACM.
- [33] Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.
- [34] Carmit Hazay and Yehuda Lindell. Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. Cryptology ePrint Archive, Paper 2009/045, 2009.
- [35] David Jao, Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Aaron Hutchinson, Amir Jalali, Koray Karabina, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Geovandro Pereira, Joost Renes, Vladimir Soukharev, and David Urbanik. Supersingular isogeny key encapsulation, 2017. <https://sike.org/>.
- [36] Stanislaw Jarecki, Ali Bagherzandi, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. Cryptology ePrint Archive, Paper 2010/561, 2010.
- [37] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and t-pake in the password-only model. Cryptology ePrint Archive, Paper 2014/650, 2014.
- [38] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Topps: Cost-minimal password-protected secret sharing based on threshold oprf. Cryptology ePrint Archive, Paper 2017/363, 2017.
- [39] Stanisław Jarecki and Xiaomin Liu. Efficient oblivious pseudorandom function with applications to adaptive ot and secure computation of set intersection. 2009.
- [40] Stanisław Jarecki and Xiaomin Liu. Fast secure computation of set intersection. 2010.

- [41] Novak Kaluđerović, Thorsten Kleinjung, and Dušan Kostić. Cryptanalysis of the generalised Legendre pseudorandom function. volume 4, pages 267–282, 2020.
- [42] Novak Kaluđerović, Thorsten Kleinjung, and Dusan Kostic. Improved key recovery on the legendre prf. Cryptology ePrint Archive, Paper 2020/098, 2020.
- [43] Marcel Keller. Mp-spdz: A versatile framework for multi-party computation. Cryptology ePrint Archive, Paper 2020/521, 2020.
- [44] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Mascot: Faster malicious arithmetic secure computation with oblivious transfer. Cryptology ePrint Archive, Paper 2016/505, 2016.
- [45] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious prf with applications to private set intersection. Cryptology ePrint Archive, Paper 2016/799, 2016.
- [46] Ueli Maurer. Secure multi-party computation made simple. *Discrete Applied Mathematics*, 154:370–381, 2006.
- [47] Payman Mohassel and Peter Rindal. Aby3: A mixed protocol framework for machine learning. Cryptology ePrint Archive, Paper 2018/403, 2018. <https://eprint.iacr.org/2018/403>.
- [48] Payman Mohassel and Yupeng Zhang. Secureml: A system for scalable privacy-preserving machine learning. Cryptology ePrint Archive, Paper 2017/396, 2017. <https://eprint.iacr.org/2017/396>.
- [49] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. Blake3, one function fast everywhere, 2020.
- [50] Seiichiro Tani. Claw finding algorithms using quantum walk. *Theoretical Computer Science*, 410(50):5285–5297, Nov 2009.
- [51] Wim van Dam, Sean Hallgren, and Lawrence Ip. Quantum algorithms for some hidden shift problems, 2002.
- [52] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. FALCON: honest-majority maliciously secure framework for private deep learning. *CoRR*, abs/2004.02229, 2020.
- [53] Thomas Wu. The SRP authentication and key exchange system. RFC, Internet Engineering Task Force, 2000. <https://www.rfc-editor.org/rfc/rfc2945>.

Appendix A.

Generating secret shares locally

Our main protocols use secret shares of random values and secret shares of zero in addition to the main inputs. The reason for using these auxiliary elements is to provide a layer of security by masking intermediate calculations. We address the generation of random/zero secret shares in this Appendix.

The methods we use are standard, and we provide them in our terminology for completeness. We urge the interested reader to find an in-depth analysis of generating secret shared correlated randomness for general adversarial structures with proofs of security in the work by Gilboa et al. [31].

A secret sharing of a random field element among the servers can be computed in an information-theoretically secure way with inter-server communication. More practical protocols are possible if we allow the security to be reduced from information-theoretic to computational. As our protocols are already assuming computational security, this is a trade-off we are happy to make. Additionally, we assume that all parties agree on a public pseudorandom function $F_k(x)$ (to be used for generating randomness locally) and on a seed s , shared as an ASS/RSS/DRSS secret sharing. More precisely it is the shares of the secret sharing of s which constitute the seed. The seed itself can be thought of as agreed upon at the beginning of the

protocol or obtained through ASS/RSS/DRSS based non-interactive key exchange (NIKE). At each generation of a new secret sharing, the PRF is evaluated at the seed and some input x which is assumed to be known to all parties. This can be a counter, timestamp, some public id, etc.

Note that agreeing on the PRF, the seed and the inputs can be done once at the start of the setup. Afterwards new secret shares can be generated on-the-go. In general these elements are generated during an offline phase and stored for use during the on-line phase of the protocol.

Let $\dot{T} \subseteq \{1, \dots, n\}$ (ASS) or $\dot{T} = \mathcal{T}$ (RSS) or $\dot{T} = \mathcal{T}^2$ (DRSS) be a set of share indices.

A.1. Generating random secret shares locally

Let $(s_{\dot{T}})_{\dot{T} \in \dot{\mathcal{T}}}$ be a random seed shared among the servers as ASS/RSS/DRSS. Let x be some public input known to all servers.

The parties compute their share of the new random secret share as:

$$r_{\dot{T}} = F_{s_{\dot{T}}}(x),$$

and the secret shared element is $r := \sum_{\dot{T} \in \dot{\mathcal{T}}} r_{\dot{T}}$.

A.2. Generating secret shares of zero locally

First, we recall how to compute a secret share of zero, assuming any pair of parties holding shares hold a pair of correlated random elements. Then, we show how to compute such random elements from shared seeds.

Assume that for each pair of shares $\dot{T}_1 \neq \dot{T}_2 \in \dot{\mathcal{T}}$ the servers holding $\dot{T}_1$ or $\dot{T}_2$ hold a 2-party additive secret share of zero known only to them, i.e., $r_{\dot{T}_1 \dot{T}_2}$ and $r_{\dot{T}_2 \dot{T}_1} \in \mathbb{F}_p$ such that

$$r_{\dot{T}_1 \dot{T}_2} + r_{\dot{T}_2 \dot{T}_1} = 0.$$

The servers can use them to compute a valid secret sharing of zero

$$r_{\dot{T}} := \sum_{\dot{T}_2 \in \dot{\mathcal{T}} \setminus \{\dot{T}\}} r_{\dot{T} \dot{T}_2}.$$

Furthermore:

$$\sum_{\dot{T} \in \dot{\mathcal{T}}} r_{\dot{T}} = \sum_{\dot{T} \in \dot{\mathcal{T}}} \sum_{\dot{T}_2 \in \dot{\mathcal{T}} \setminus \{\dot{T}\}} r_{\dot{T} \dot{T}_2} = \frac{1}{2} \sum_{\dot{T} \neq \dot{T}_2 \in \dot{\mathcal{T}}} r_{\dot{T} \dot{T}_2} + r_{\dot{T}_2 \dot{T}} = 0.$$

Therefore, the secret sharing $(r_{\dot{T}})_{\dot{T} \in \dot{\mathcal{T}}}$ is a secret sharing of 0.

What is left is to show how to compute $r_{\dot{T}_1 \dot{T}_2}$ and $r_{\dot{T}_2 \dot{T}_1}$. Let “ $<$ ” be a strict total order on $\dot{\mathcal{T}}$, and assume that for each $\dot{T}_1 \neq \dot{T}_2 \in \dot{\mathcal{T}}$ there is a seed $s_{\{\dot{T}_1, \dot{T}_2\}}$ known only by servers holding $\dot{T}_1$ or $\dot{T}_2$. Then, the parties can generate the two-element secret shares of zero as:

$$r_{\dot{T}_1 \dot{T}_2} = (-1)^{(\dot{T}_1 < \dot{T}_2)} F_{s_{\{\dot{T}_1, \dot{T}_2\}}}(x),$$

where $(-1)^{(\dot{T}_1 < \dot{T}_2)} = 1$ if $(\dot{T}_1 < \dot{T}_2)$ and -1 otherwise.

The seeds $s_{\{\dot{T}_1, \dot{T}_2\}}$ can also be obtained from a NIKE [29] protocol. Assume that the parties hold secret keys $(sk_{\dot{T}})_{\dot{T} \in \dot{\mathcal{T}}}$ according to the corresponding secret sharing scheme, and that all the corresponding public keys $pk_{\dot{T}}$ are

public and authenticated. Then, the servers can compute the seeds as:

$$\begin{aligned} s_{\{T_1, T_2\}} &= \text{NIKE.shared_key}(sk_{T_1}, pk_{T_2}) \\ &= \text{NIKE.shared_key}(sk_{T_2}, pk_{T_1}), \end{aligned}$$

where a post-quantum NIKE could be used, for example CSIDH [14].

A.3. RSS to RSS Multiplication Protocol and Proof

Protocol 4 Malicious RSS to RSS multiplication	
Input :	$\llbracket a \rrbracket_R, \llbracket b \rrbracket_R$, RSS shares, $\llbracket s \rrbracket_D$, DRSS share of a random seed.
Output:	$\llbracket ab \rrbracket_R$.
Players: Server $S_1, \dots, S_n$.	
Input :	$(a_T, b_T)_{T \in \mathcal{T}_i} \subseteq \mathbb{F}_p$ RSS shares of a, b $(s_{T_1 T_2})_{T_1, T_2 \in \mathcal{T}_i} \subseteq \mathbb{F}_p$ shares of DRSS share of random seed
<pre> 1 for $T_1, T_2 \in \mathcal{T}_i$ do 2 $c^{i, T_1 T_2} = a_{T_1} b_{T_2}$ 3 Compute $\llbracket c^{i, T_1 T_2} \rrbracket_R = (c_T^{i, T_1 T_2})_{T \in \mathcal{T}} \leftarrow$ RSS($c^{i, T_1 T_2}; s_{T_1 T_2}$) 4 for $j \in \{1, \dots, n\} \setminus \{i\}$ do 5 Send $((c_T^{i, T_1 T_2})_{T \in \mathcal{T}_j})_{T_1, T_2 \in \mathcal{T}_i}$ to server j </pre>	
<pre> 6 Upon receiving $((c_T^{j, T_1 T_2})_{T \in \mathcal{T}_i})_{T_1, T_2 \in \mathcal{T}_j}$ from server j 7 for $T_1, T_2 \in \mathcal{T}$ do 8 for $T \in \mathcal{T}_i$ do 9 for $j, k \in \mathcal{S}_{T_1 T_2}$ do 10 if $c_T^{j, T_1 T_2} \neq c_T^{k, T_1 T_2}$ then $\perp$; 11 $c_T^{T_1 T_2} := c_T^{j, T_1 T_2}$ for any $j \in \mathcal{S}_{T_1 T_2}$ 12 for $T \in \mathcal{T}_i$ do 13 $(ab)_T := \sum_{T_1, T_2 \in \mathcal{T}} c_T^{T_1, T_2}$ 14 return $((ab)_T)_{T \in \mathcal{T}_i}$ </pre>	

Theorem 4. *Protocol 4 securely realizes $\mathcal{F}_{\text{Mul}}$ against up to $t < n/3$ malicious servers.*

Proof. By abuse of notation we denote with $\mathcal{A}$ both the adversary and the set of indices of corrupted servers, while we denote with $\mathcal{H} := \{1, \dots, n\} \setminus \mathcal{A}$ the set of indices of honest clients. Similarly we define $\mathcal{T}_{\mathcal{A}} := \{T \in \mathcal{T} \mid \mathcal{A} \cap T = \emptyset\}$ and $\mathcal{T}_{\mathcal{H}} := \mathcal{T} \setminus \mathcal{A}$. Note that $\mathcal{T}_{\mathcal{H}} \neq \{T \in \mathcal{T} \mid \mathcal{H} \cap T = \emptyset\}$. In other words $\mathcal{T}_{\mathcal{A}}$ is the set of shares owned by the adversary, and $\mathcal{T}_{\mathcal{H}}$ the set of shares not owned by the adversary. In particular $\#\mathcal{T}_{\mathcal{H}} \geq 1$.

The correctness of the algorithm follows from a careful analysis of indices.

We argue that it is enough to prove privacy in the semi-honest case. The protocol consists of one round of communication, so the only way for the adversary to act maliciously is to send a wrong share $c_T^{T_1 T_2}$ to an honest client. However, since there is always a client $i \in \mathcal{H}$ holding T_1, T_2 , the correct share will also be sent to the

honest client by server i , and the honest client will return $\perp$. This proves the soundness of the algorithm.

In the malicious case, we can construct a simulator as follows: The simulator, upon receiving $((c_T^{T_1 T_2})_{T \in \mathcal{T}})_{T_1, T_2 \in \mathcal{T}_{\mathcal{A}}}$ from $\mathcal{A}$, generates random field elements $((r_T^{T_1 T_2})_{T \in \mathcal{T}})_{(T_1, T_2) \in \mathcal{T} \times \mathcal{T}_{\mathcal{H}} \cup \mathcal{T}_{\mathcal{H}} \times \mathcal{T}}$, and set $r_T^{T_1 T_2} = c_T^{T_1 T_2}$ for $T_1, T_2 \in \mathcal{T}_{\mathcal{A}}$. They send $((r_T^{T_1 T_2})_{T \in \mathcal{T}_i})_{T_1, T_2 \in \mathcal{T}}$ to each $i \in \mathcal{A}$.

For each T_1, T_2 , the elements that the adversary receives are RSS shares of $c^{T_1 T_2} = a_{T_1} b_{T_2}$. Either the adversary knows $c^{T_1 T_2}$, so they receive the same shares that they already know, or the adversary does not know both T_1, T_2 , so they receive random elements $r_T^{T_1 T_2}$. But these random elements are indistinguishable from actual shares $(c_T^{T_1 T_2})_{T \in \mathcal{T}} \leftarrow \text{RSS}(c^{T_1 T_2}, s_{T_1 T_2})$ to any adversary not holding all shares. $\square$